

北京邮电大学

大学生创新训练计划项目 结题报告

项目编号: 202503053 是否参加 2026 鸿雁杯比赛: 否

项目类别: 导师科研类☒ 自主探索类☐
校企合作类☐ 滚动支持类☐ 探索课堂☐
雏雁获奖类☐ 科创融合大挑战☐ 国际专项☐

项目中文名称: 星轨 (StarTrack): 面向自然语言交互的 WiFi-视觉融合 AR 室内导航系统

项目名称 (英文): StarTrack: WiFi-Visual Fusion AR Indoor Navigation System for Natural Language Interaction

项目依托学院: 未来学院

项目负责人: 李同宇

联系电话: 15143366635

E-Mail: 3858667251@qq.com

指导教师: 王吴凡

E-Mail: wufanwang@bupt.edu.cn

1、 2026 年 6 月 19 日

项目基本情况

项目名称						
项目主要研究成员	序号	姓名	学号/工号	专业/职称	学院	项目分工/指导内容
	1	李同宇	2024212731	计 算 机 类 (元班)	未来学院	项目负责人/系统架构设计/ 前端AR 与后端路径规划
	2	李家祥	2024212739	计算机类 (元班)	未来学院	WiFi 定位与后端
	3	邢铭航	2024212707	计算机类 (元班)	未来学院	视觉定位与后端
	4	王宇馨	2024210836	电子信息科学与技术	电子工程学院	项目文档撰写
	5					
指导教师	1	王吴凡	2010815084	副研究员	计算机学院	项目全流程指导
	2					

一、项目执行情况简介

(一) 项目研究的目的

本项目面向大型室内空间中“无 GPS 可用、二维地图难理解、传统室内定位精度不足”的实际问题，研究并实现一套低成本、高精度、交互直观的 AR 室内导航系统。系统以建筑内已有 WiFi 网络和普通手机摄像头为主要感知来源，通过 WiFi 指纹粗定位、HLoc 视觉精定位、语义拓扑寻路和 Unity AR 渲染的协同工作，完成从用户输入目的地到实景箭头导航的完整闭环。

项目的核心目标包括：一是复用现有 WiFi 网络形成区域级定位先验，降低专用基站部署成本；二是利用视觉定位获得用户在室内空间中的精确位置与朝向，支撑亚米级导航；三是通过稀疏语义拓扑图完成可通行路径规划，避免简单直线导航无法绕开书架、墙体和楼梯等问题；四是通过 AR 箭头直接叠加在摄像头画面中，降低用户在二维地图与真实空间之间转换方向的认知负担；五是进一步支持自然语言目的地表达，使用户可以用“找一个安静学习的位置”“去讨论区”等方式发起导航。

(二) 成果的主要内容

项目执行期间，团队按照“需求分析—数据采集—离线建图—后端定位服务—路径规划—移动端 AR 展示—实景测试”的路线推进，完成了 StarTrack 系统原型及配套数据、算法和文档成果。

第一，完成 WiFi 指纹粗定位模块。团队基于图书馆实测场景构建 RadioMap 数据，记录 AP、BSSID、RSSI 及采样点坐标信息，并完成异常值处理、同点信号合并和结构化存储。在线阶段，系统将 WKNN 算法封装为后端服务，根据手机上传的 WiFi 扫描结果估计用户所在区域，为后续视觉定位提供空间先验。

第二，完成 HLoc 视觉精定位模块。团队在图书馆场景中采集视频和参考图像，完成图像抽帧、特征提取、图像匹配与 COLMAP 三维重建，形成可用于定位的视觉地图。在线阶段，系统通过 NetVLAD 检索候选图像，结合 SuperPoint、LightGlue、PnP 与 RANSAC 求解查询图像的六自由度相机位姿，从而获得用户位置和朝向信息。测试结果表明，视觉定位模块能够达到亚米级定位精度，满足室内书架、功能区等细粒度导航需求。

第三，完成 Zone 约束式候选筛选机制。系统将 WiFi 粗定位结果映射为一个或多个 Zone 标签，并把该标签作为视觉检索阶段的候选范围约束。该机制的重点不再是简单叠加两类定位结果，而是让 WiFi 结果提前参与视觉检索空间筛选，减少重复书架、相似走廊等场景下的跨区域误匹配风险，同时降低视觉检索与匹配的计算开销。经过多轮实测调整后，该机制对系统响应速度和工程稳定性起到了重要作用。

第四，完成稀疏语义拓扑图与路径规划模块。团队从 COLMAP 三维模型和实际空间结构中选取走廊交汇点、区域入口、楼梯口、电梯口、学习区和讨论区等关键位置作为拓扑节点，记录节点坐标、语义名称、楼层信息和连通关系，并以 JSON 形式维护。在线导航时，系统根据用户当前位置和目的地节点，使用 Dijkstra 算法生成可通行路径，输出关键节点序列供前端渲染。

第五，完成自然语言目的地解析与多模态语义节点扩展。项目在拓扑节点中加入别名、文字描述、参考图像和 CLIP 语义向量，使节点不再只是几何坐标，而是同时具备空间位置、视觉证据和语义含义。系统既支持标准地点名称匹配，也支持自然语言意图解析，能够把用户输入映射到具体可导航节点，提升了系统交互的自然性和智能化程度。

第六，完成 Unity 移动端 AR 导航原型和端云协同闭环。移动端负责目的地输入、WiFi 扫描、ARCamera 图像采集、请求发送和 AR 箭头渲染；后端负责 WiFi 定位、视觉定位、语义解析和路径规划。系统通过 HTTP 接口传输图像、WiFi 扫描结果和路径 JSON，并完成 COLMAP 坐标系到 Unity AR 空间的坐标对齐。在图书馆实景多次测试中，从用户发起导航请求到 AR 箭头出现通常约 5 至 6 秒，能够满足按需触发式室内导航体验。

综合来看，本项目完成了 WiFi 指纹库、视觉三维模型、Zone 空间约束、语义拓扑图、自然语言目的地解析、后端定位与寻路服务以及 Unity 移动端 AR 导航原型等主要成果，形成了完整的技术文档、实验数据、源码和演示材料。项目验证了利用已有 WiFi 网络与普通移动终端构建低成本、高精度 AR 室内导航系统的技术可行性，也为后续向智慧校园、图书馆导览、公共建筑导航等场景扩展提供了基础。

附：主要成果

研究成果名称	成果形式	项目参与者姓名	学院、专业	学号	任务分工
StarTrack 移动端 AR 导航原型	软件原型 / APK	李同宇, 李家祥, 邢铭航	未来学院、计算机类（元班）	2024212731	Unity 前端、AR 箭头渲染、导航交互与实景测试
WiFi 指纹粗定位与 RadioMap 数据	算法模块 / 数据集	李家祥	未来学院、计算机类（元班）	2024212739	WiFi 采集、WKNN 定位、区域先验生成
HLoc 视觉定位与 COLMAP 建图流程	算法模块 / 视觉地图	邢铭航	未来学院、计算机类（元班）	2024212707	图像采集、特征匹配、三维重建与位姿解算
语义拓扑图、自然语言目的地解析与路径规划服务	后端服务 / 数据结构	李同宇	未来学院、计算机类（元班）	2024212731	拓扑节点标注、CLIP 语义索引、Dijkstra 路径规划

论文发表情况：暂无正式发表论文。项目已形成完整设计文档、源码、实验记录、演示视频和移动端原型；后续拟围绕 WiFi-视觉分层协同定位、Zone 候选约束和多模态语义拓扑导航继续整理技术报告或论文成果。

创新（特色）与应用

本项目的创新与特色主要体现在以下三个方面。

第一，提出 WiFi 先验约束下的视觉定位协同机制。传统融合方案往往把 WiFi 和视觉作为两个独立定位结果再进行后验合并，而本项目将 WiFi 粗定位结果转化为视觉检索前的空间先验。系统先通过 RadioMap 和 WKNN 算法获得用户所在区域，再映射为 Zone 标签，约束 NetVLAD 候选图像检索范围。该机制使 WiFi 不只是“粗略定位工具”，而成为视觉定位流程中的前置筛选条件，在图书馆书架和走廊纹理高度相似的场景中，有助于减少跨区域误匹配并提升在线响应速度。

第二，构建“几何坐标+视觉证据+语义描述”的多模态拓扑图。项目没有直接在庞大的三维点云上寻路，而是将真实室内空间抽象为包含几十个关键节点的稀疏拓扑图。每个节点不仅保存三维坐标和连通关系，还绑定参考图像、别名、文字描述和语义向量，使系统能够把用户输入的地点名称或自然语言意图映射为可寻路节点。这样，系统不仅能回答“我在哪里”，还能进一步回答“我想去哪”和“应该怎么走”。

第三，实现端云协同与 AR 可视化结合的完整工程闭环。移动端只承担 WiFi 扫描、图像采集、目的地输入和 AR 渲染，把视觉检索、特征匹配、位姿解算、语义检索和路径规划等复杂计算交给后端完成，降低了手机端算力和功耗压力。后端返回路径 JSON

后，Unity 前端通过坐标系对齐把路径节点映射到 AR 空间，在摄像头实景画面中显示三维箭头，实现“所见即路径”的导航体验。

本项目已在图书馆场景中完成原型验证，可引导用户前往学习区、讨论区、楼梯口、卫生间等功能位置。多次实测表明，系统在无需额外部署 UWB 基站、蓝牙信标或激光雷达等专用定位设备的情况下，实现了亚米级视觉定位和约 5 至 6 秒的端到端 AR 导航响应，验证了整体方案的可用性和工程落地价值。应用方面，系统可进一步扩展至教学楼、医院、商场、博物馆、办公园区和地下停车场等复杂室内空间，为智慧校园、智慧场馆和公共建筑数字化服务提供低成本、可复制的技术方案。

（一）项目研究进程说明

本项目围绕复杂室内环境中的高精度定位与 AR 导航需求，按照“需求分析与技术调研—总体方案设计—离线数据构建—核心模块开发—系统集成与实测优化—成果总结”的路线推进。

项目初期，团队首先对图书馆、教学楼、医院和商场等典型室内场景进行需求分析，明确用户在陌生建筑中主要面临位置不清、方向难判断、二维地图理解成本高等问题。随后，团队对 WiFi 指纹定位、BLE 信标、UWB、视觉定位、IMU/PDR 和 LiDAR 等方案进行了对比，确定采用“已有 WiFi 网络提供低成本区域先验，视觉定位提升精度，语义拓扑图完成路径规划，Unity 实现 AR 导航显示”的总体技术路线。

方案设计阶段，团队将系统划分为数据层、算法服务层、业务逻辑层和用户交互层。移动端负责 WiFi 扫描、图像采集、目的地输入和 AR 渲染，后端负责 WiFi 定位、视觉定位、自然语言目的地解析和路径规划，形成端侧轻量采集与显示、后端集中计算的协同模式。同时，团队提出利用 WiFi 粗定位结果生成 Zone 区域先验，用于约束视觉候选图像检索范围，以应对图书馆书架和走廊纹理重复带来的视觉误匹配问题。

离线数据构建阶段，团队完成图书馆场景下的 WiFi 指纹采集、坐标标定、异常值过滤和 RadioMap 结构化处理；在视觉数据方面，按照走廊双向覆盖、区域内部闭环和关键位置补拍的方式采集图像与视频，完成抽帧、特征提取、图像配对和 COLMAP 三维重建，建立视觉定位模型。随后，团队依据实际空间结构和导航需求选取关键节点，构建包含坐标、楼层、名称、别名和连通关系的稀疏语义拓扑图。

核心算法开发阶段，团队将 WKNN 算法封装为 Python 后端服务，实现 WiFi 指纹向量生成、信号相似度计算和加权位置估计；基于 HLoc 框架集成 NetVLAD、SuperPoint、LightGlue、PnP 和 RANSAC，完成查询图像的候选检索、局部特征匹配和六自由度位姿解算；基于 Dijkstra 算法完成当前位置到目的地节点的最短可行路径计算。在此基础上，项目进一步加入自然语言目的地解析能力，为拓扑节点绑定参考图像、别名、文字描述和语义向量，使用户可以通过更自然的表达发起导航。

系统集成阶段，团队搭建后端导航接口，将 WiFi 粗定位、Zone 映射、视觉精定位、语义目的地解析和路径规划串联为一次完整请求，并统一以 JSON 格式完成前后端数据交互。移动端使用 Unity 开发，实现 WiFi 扫描、ARCamera 图像采集、导航请求发送、结果接收和 AR 箭头渲染。针对 COLMAP 坐标系与 Unity AR 空间不一致的问题，

团队完成了相机位姿转换、尺度对齐和路径节点映射，使三维箭头能够叠加到摄像头实景画面中的对应方向。

项目后期，团队在图书馆场景中进行了多轮实测，围绕 WiFi 定位结果、视觉定位稳定性、Zone 候选约束、端到端响应时间、路径正确性、自然语言目的地解析和 AR 显示效果持续调试。实测结果表明，系统能够实现亚米级视觉定位效果，从用户点击导航到 AR 箭头出现通常约 5 至 6 秒，基本完成从“输入导航需求”到“实景路径引导”的闭环。根据测试反馈，团队也对系统进行了必要调整：一方面保留 Zone 机制在检索加速和误匹配抑制方面的作用，另一方面认识到 Zone 划分粒度会影响视觉定位表现，后续需要进一步研究更自适应的区域划分方式。

项目最终形成了移动端 AR 导航原型、后端定位与路径规划服务、WiFi 指纹库、视觉建图数据、语义拓扑图、自然语言目的地解析模块、实验记录、项目设计文档和演示材料。项目过程中，团队成员围绕前端 AR、后端服务、WiFi 定位、视觉定位、拓扑规划、文档整理和测试验证进行分工协作，并通过阶段性联调和代码同步保证系统各模块能够稳定衔接。

二、研究总结报告

（一）项目取得的主要成绩和收获

本项目围绕复杂室内环境中的高精度定位与直观导航需求，完成了星轨（StarTrack）室内高精度导航系统的设计、开发与原型验证。项目以既有 WiFi 网络作为区域级感知基础，将 WiFi 指纹粗定位、Zone 区域约束、视觉精定位、稀疏语义拓扑图路径规划、自然语言目的地解析和 Unity AR 导航显示等模块进行集成，形成了从用户输入导航需求、移动端采集数据、后端定位计算到路径生成与实景引导的完整技术闭环。

在 WiFi 定位方面，团队完成了目标场景的 WiFi 指纹采集、坐标标定、数据清洗和 RadioMap 构建，并将 WKNN 算法封装为后端标准定位服务。该模块能够根据移动端实时上传的 AP 与 RSSI 数据，快速输出用户所在楼层及二维平面粗坐标，为后续视觉定位提供区域级空间先验。项目实践表明，利用建筑内已有 WiFi 网络可以在不额外部署专用定位设备的情况下获得较稳定的初始位置估计，为低成本室内导航提供了可行基础。

在视觉定位方面，团队基于 hloc 框架完成了图书馆场景的视频采集、图像抽帧、特征提取、图像匹配和 COLMAP 三维重建，形成了包含参考图像、三维特征点、相机位姿及特征索引的视觉地图。在线定位过程中，系统依次使用 NetVLAD、SuperPoint、LightGlue 以及 PnP 与 RANSAC 完成候选检索、特征匹配和六自由度位姿求解，实现了对用户精确位置与朝向的估计。

针对图书馆书架、走廊等区域视觉纹理高度重复、全局检索容易发生跨区域误匹配的问题，项目实现了 Zone 约束式空间筛选机制。系统先根据 WiFi 粗定位结果确定用户所在的一个或多个 Zone，再将该结果作为 Prefix 约束传递给视觉定位模块，使视觉检索仅在相应区域的图像子集中进行。该方法将 WiFi 定位结果由独立的位置估计转化为视觉检索的空间先验，显著缩小了候选范围，提高了定位效率和复杂场景下的稳定性。

在路径规划方面，团队将 COLMAP 视觉模型中的关键空间位置提炼为稀疏语义拓扑节点，完成了区域入口、走廊交汇点、楼梯、电梯及重要功能位置的节点标注与连通关系构建，并使用 Dijkstra 算法生成从用户当前位置到目标位置的最短可行路径。与直接在大规模三维点云上进行路径搜索相比，稀疏语义拓扑图具有数据规模小、语义清晰、计算效率高和便于维护更新等特点。

项目进一步扩展了自然语言目的地解析功能。团队为拓扑节点增加参考图像、别名和语义描述等信息，使节点由单一几何坐标扩展为包含空间信息、视觉信息和语言描述的多模态节点。用户既可以输入标准地点名称，也可以使用“寻找安静学习的位置”“前往最近的卫生间”等自然语言表达导航意图，系统再将用户需求转换为对应的目标节点，为提升室内导航交互的自然性提供了初步技术基础。

在终端实现方面，项目完成了基于 Unity 的移动端 AR 导航原型。移动端负责 WiFi 扫描、图像采集、目标输入和 AR 显示，后端负责 WiFi 定位、视觉特征推理、位姿求解、语义解析和路径规划。系统通过 HTTPS 实现端云数据交互，并通过坐标系转换将后端返回的路径节点映射到 Unity 空间，在摄像头实景画面中显示三维箭头和目标提示。实景测试表明，系统能够实现亚米级定位精度和 5 秒以内的端到端响应，验证了总体技术路线的可行性。

通过项目实施，团队成员在专业知识、科研训练和工程实践方面均取得了较大收获。首先，团队进一步掌握了 WiFi 指纹定位、计算机视觉、三维重建、图论路径规划、后端服务和增强现实等相关理论与方法，实现了对不同课程知识的综合运用。其次，团队经历了从需求分析、技术调研、数据采集、算法复现到系统集成和实景测试的完整过程，提升了独立解决复杂工程问题的能力。再次，团队在项目分工、接口协商、版本管理、实验记录、文档撰写和成果汇报等方面积累了实践经验，增强了团队协作与项目管理能力。

总体来看，本项目基本完成了预期研究任务，形成了较完整的系统原型、场景数据、算法模块、技术文档和实验结果。项目不仅验证了 WiFi 与视觉异构融合用于室内高精度导航的可行性，也为后续开展多场景测试、系统性能优化、知识产权申请和成果推广奠定了基础。

（二）项目工作中的困难、问题和建议

在项目实施过程中，团队也遇到了相关技术和工程方面的困难：

1. 离线场景建库工作量较大。系统正式运行前需要完成 WiFi 指纹采集、视觉视频采集、图像抽帧、三维模型重建、Zone 划分和拓扑节点标注等工作。虽然这些数据能够在运行阶段重复使用，但对于面积较大、楼层较多的建筑，前期采集和标注仍需要投入较多时间。当场景内部出现装修、书架移动或通道调整时，还需要同步更新 RadioMap、参考图像库和拓扑图。后续可进一步开发自动化采集、增量建图、场景变化检测和拓扑节点自动生成工具，以降低场景部署与维护成本。

2. 视觉定位容易受环境条件影响。图书馆中存在大量相似书架和重复走廊，同时光照变化、人员遮挡、运动模糊及摄像头视角差异也会影响视觉特征提取和匹配效果。项目通过 Zone 约束、RANSAC 异常点过滤和 WiFi—视觉一致性校验降低了部分误匹配风险，但在强逆光、低照度、大面积遮挡或纹理不足的场景中，定位成功率和初始化速度仍有提升空间。后续可引入更鲁棒的视觉特征模型、图像增强、IMU 辅助定位和连续跟踪机制，提高系统在复杂动态环境中的适应能力。

3. 端到端响应时间受视觉模型推理限制。WiFi 定位和拓扑路径规划的计算时间较短，而 NetVLAD 特征计算、LightGlue 匹配和 PnP 位姿求解占据了系统大部分处理时间。虽然目前系统已将端到端响应控制在 5-6 秒左右，但与连续实时导航仍存在一定差距。后续可通过模型量化、ONNX Runtime 与 TensorRT 加速、GPU 批处理、候选图像缓存以及端侧轻量特征提取等方式进一步缩短响应时间。

4. 当前验证场景相对有限。项目主要在学校图书馆环境中开展数据采集、系统开发和实景测试。图书馆能够较好体现书架密集、纹理重复和走廊结构复杂等典型问题，但医院、商场、机场等场景在人流密度、建筑规模、网络环境和使用需求方面存在较大差异。系统在多楼层切换、多用户并发、弱网传输及长期运行条件下的性能仍需通过更多场景进一步验证。

5. 自然语言意图导航功能仍处于初步探索阶段。当前系统可以利用地点别名、文字描述和多模态语义信息将部分自然语言需求映射到目标节点，但面对更加模糊、复杂或包含多项条件的表达时，仍可能出现理解不准确的问题。后续可进一步完善节点语义描述，扩大自然语言表达样本，引入更适合中文室内场景的语言模型或视觉语言模型，并结合距离、区域属性和开放状态进行综合目的地推荐。

6. 模块间的坐标、数据格式和接口约定复杂。WiFi 定位使用二维物理坐标，视觉定位输出 COLMAP 坐标系下的六自由度位姿，Unity 前端则使用自身的三维坐标系。任何一个坐标转换环节出现误差，都可能导致路径偏移或 AR 箭头位置不准确。团队通过统一 JSON 数据结构、编写坐标转换函数和增加日志输出完成了问题排查，但今后还应进一步加强接口文档、自动化测试和模块级单元测试。

针对上述问题，团队提出以下建议：

1. 项目初期应尽早确定统一的数据格式、坐标系统和接口规范，减少后期系统联调成本。
2. 加强过程性实验记录，保留采集参数、模型版本、测试地点和异常情况，保证实验结果可追溯；
3. 合理安排算法研究与工程实现的时间比例，在保证核心技术创新的同时重视移动端体验和系统稳定性；
4. 建议学校继续为大创项目提供实验场地、服务器算力、跨学院技术交流和成果转化方面的支持，帮助学生项目从原型验证进一步走向真实应用。

项目仍存在建库成本、环境适应性和多场景验证不足等问题，但这些问题也为后续研究提供了明确方向。团队将继续围绕定位连续性、视觉算法加速、自动化建图、多模态语义理解和跨场景部署开展优化，进一步提高系统的稳定性、实时性与实际应用能力。

其他附件：大创项目报告书

星轨 (StarTrack) : 面向自然语言交互的 WiFi- 视觉融合 AR 室内导航系统

**StarTrack: WiFi-Visual Fusion AR Indoor Navigation System
for Natural Language Interaction**

大创项目报告书

目录

第一章 目标问题与意义价值	4
1.1 研究背景	4
1.2 现有室内导航技术的对比分析	5
1.3 项目目标	7
1.4 项目挑战	7
第二章 设计思路与方案	9
2.1 技术路线与功能闭环	9
第三章 方案实现	12
3.1 软件架构与模块分层	12
3.2 部署架构：“端-云”协同	13
3.3 链路运转流程	14
3.4 终端用户使用流程	15
3.5 基于指纹库的 WiFi 粗定位模块	16
3.5.1 离线采集端重构：Anyplace Logger 采集 WiFi 指纹数据与建立 Radiomap	16
3.5.2 在线匹配算法实现	18
3.6 高精度分层视觉定位模块	19
3.6.1 离线稠密重建：分区视频流采样与三维点云生成	20
3.6.2 快速精确定位	22
3.7 Prefix 约束式空间筛选机制	23
3.7.1 Zone 区域先验与检索空间降维	24
3.8 基于稀疏语义拓扑图的寻路算法	25
3.8.1 物理轨迹提纯：从稠密点云到稀疏拓扑的语义映射	25
3.8.2 在线图论求解：基于物理绝对坐标的跨楼层 Dijkstra 路径规划	27
3.8.3 多模态语义目的地解析：从自然语言到 destination_node	29
3.9 Unity 平台实现 AR 渲染与坐标系锚定	30
3.9.1 坐标系对齐的数学推导	30

3.9.2 端侧 WiFi 扫描：自研轻量级原生插件	31
3.9.3 端到端业务流程与动态路径引导	31
第四章 运行结果与应用效果	33
4.1 应用场景描述	33
4.2 核心模块性能测试与效果展示	33
4.3 系统扩展性分析	35
第五章 创新与特色	38
5.1 三大核心创新点	38
5.1.1 核心创新一：Zone 区域约束的 WiFi-视觉分层协同	38
5.1.2 核心创新二：面向自然语言意图导航的多模态语义拓扑图	38
5.1.3 核心创新三：云端计算+端侧渲染	39
5.2 项目核心优势总结	39
第六章 应用与商业价值	41
6.1 实际应用场景	41
6.2 商业价值与商业模式	42
6.3 部署与推广	42
第七章 总结与展望	44
第八章 团队与分工	44
第九章 局限性分析	46
1. 场景建库与数据维护成本仍然较高	46
2. 对环境变化与成像条件仍有一定敏感性	46
3. 当前验证范围仍以单场景原型为主	46
参考文献	47

第一章 目标问题与意义价值

1.1 研究背景

随着大型公共建筑规模的不断扩大，室内空间结构日趋复杂，用户进入建筑后往往难以快速找到目标位置，且依赖指示牌或人工问询的传统方式效率较低，难以满足实际需求。与此同时，人们在室内环境中的停留时间显著增加。相关研究表明，现代人约 80%~90% 的时间处于室内环境中。在此背景下，如何在复杂室内空间中提供类 GPS 的导航能力，逐渐成为智能空间领域的重要研究课题。

学术界与工业界早已将目光投向在室内广泛部署的无线局域网（WLAN/WiFi）。以 Anyplace 为代表的 WiFi 指纹定位方案，通过离线建立“位置-RSSI 向量”指纹库、在线做相似度匹配，实现了对既有网络基础设施的直接复用，无需引入任何额外硬件。然而，受多径传播、人体遮挡、AP 故障等物理因素制约，纯 WiFi 指纹定位的精度通常在 2~5 米，且缺乏方向信息，难以满足“精确到具体书架、具体房间”的精细导航需求。

因此，如何在既有 WiFi 网络定位的基础上，通过异构信号融合与端-边-云协同通信机制，将室内定位精度从“区域级”提升到“亚米级”，同时保持零额外基础设施投入与良好的网络传输效率，成为本项目的核心出发点。

同时，真实室内导航需求并不总是以精确地点名称出现。许多用户进入陌生建筑后，往往只能表达功能性意图，例如“找一个安静自习的位置”“我能讨论的空间”“去最近的洗手间”，而不清楚建筑内部的标准 POI 命名、区域编号或具体方位。因此，面向复杂室内空间的导航系统不仅需要回答“用户在哪里”和“应该怎么走”，还需要进一步理解“用户想去哪里”。这使自然语言意图解析与空间语义建模成为提升导航体验的重要方向，也为本项目后续引入多模态语义拓扑图提供了需求基础。

本项目的设计灵感来源于以下四点：一是尽量利用已经普及的 WiFi 基础设施，避免引入额外硬件成本；二是借助深度学习驱动的视觉定位技术弥补 WiFi 精度不足的短板；三是通过端-边-云协同通信合理分配计算与网络负载；四是通过 AR 可视化交互把路径以实景叠加箭头的方式呈现给用户，从根本上降低室

内寻路的认知负担。本项目顺应了目前多源融合定位的研究趋势，也切合产业界对低成本、可规模化室内定位方案的需求，研究价值极高。

1.2 现有室内导航技术的对比分析

为了清晰界定本项目的创新定位，下面对主流室内定位技术逐一评析，并指出其在真实大型场景中的核心局限。

BLE 蓝牙信标定位。通过部署蓝牙信标并利用终端接收到的 RSSI 做三边或加权估计。优点是功耗低、iBeacon/Eddystone 等协议成熟、终端兼容性好。但其定位精度通常在 2~5 米，难以支持“找到具体货架”这样的精细需求；更困难的是 BLE 需要信标部署密度达到 5~10 米，这意味着大型室内建筑需要部署数百甚至上千个信标，外加电池维护与固件升级，运维成本难以规模化。

UWB 超宽带定位。利用纳秒级宽带脉冲做飞行时间测量，精度可达 10~30 厘米，是近年工业场景的主流选择。但 UWB 基站单价高达数百至数千元，终端也需要 UWB 芯片（目前仅 iPhone、三星高端机型等支持），同时对基站同步与视距条件要求严格。这些限制使得 UWB 主要应用于工厂、仓储等专业场景，难以推广至商场、医院等大众消费场景。

纯 WiFi 指纹定位。以开源项目 Anyplace 为代表，离线建立“位置-RSS 向量”指纹库，在线做相似度匹配。最大优势是零额外硬件（大型建筑普遍已部署 WiFi），且智能手机原生支持 WiFi 扫描。但受多径传播、人体遮挡、AP 故障、终端差异等干扰，实际定位精度通常在 2~5 米，而且没有方向信息，用户无法获得“朝哪个方向走”的指引。Anyplace 官方在真实环境下的误差中位数为 1.96 米，这一精度对“到达某楼层或大致区域”够用，但对精细导航不足。

视觉 SLAM 与视觉定位。现代视觉定位（如 Hloc、ORB-SLAM）在良好纹理下可实现厘米级 6 自由度位姿估计，天生具备方向感知能力。但纯视觉方案有两个痛点：一是计算开销大，NetVLAD 检索+SuperGlue/LightGlue 匹配+PnP 的完整流水线在普通 CPU 上通常耗时数秒到数十秒，难以端上实时运行；二是在纹理重复的室内场景（如图书馆雷同走廊、商场相似门面）中，全局检索容易召回跨区域的错误候选，导致定位结果“瞬移”。

IMU 行人航位推算（PDR）。通过加速度计、陀螺仪、磁力计推算步长与

航向，连续更新位置。优点是不依赖外部信号、延迟极低。但 PDR 的根本缺陷在于误差随时间累积，几分钟内误差就可达几十米，不能单独使用，必须依靠其他源定期校正。

LiDAR 或深度相机 SLAM。LiDAR 和 ARKit 等方案精度高、鲁棒性强，但 LiDAR 硬件价格居高不下；ARKit/ARCore 当前只提供局部跟踪能力，没有“建筑级”的定位和路径规划接口。

综上，现有室内定位技术主要面临三方面共性问题：一是精度与成本难以兼顾，高精度方案依赖专用硬件与网络，而低成本方案精度有限；二是单一模式易受环境因素影响，难以在复杂场景中保持稳定性能；三是缺乏直观的导航呈现方式，用户仍需进行“地图到现实”的认知转换。

表 1 现有室内方法对比

方案	是否依赖专用网络	是否利用 WiFi	端-边-云协同能力	典型精度	终端成本	直观视觉指导	综合评价
BLE 信标	是（专用信标网络）	否	弱	2~5 m	信标数百元/个	否	维护成本高
UWB	是（专用基站网络）	否	弱	10~30 cm	基站数千元/套	否	硬件依赖强
纯 WiFi 指纹	否	是	中	2~5 m	零	否	精度受限
纯视觉定位	否	否	弱	亚米级	零	否	检索空间爆炸
IMU/PDR	否	否	弱	累积漂移	零	否	无法独立使用
LiDAR/ARKit	是（专用传感器）	否	弱	厘米级	数千元	否	规模化困难

方案	是否依赖专用网络	是否利用 WiFi	端-边-云协同能力	典型精度	终端成本	直观视觉指导	综合评价
本项目 StarTrack	否	是(深度利用)	强(端-边-云三级)	亚米级	零	是	兼顾全部指标, 支持意图导航

表一 室内定位方法对比

针对上述问题, 本项目从多源信息融合与空间约束的角度出发, 设计一种兼顾低成本、高精度与良好用户体验的室内导航方案。

1.3 项目目标

针对上述背景和现有方案的局限, 本项目设定了以下五项明确目标:

目标一: 亚米级定位精度。在真实大型室内场景下实现平均误差小于 1 米的绝对定位, 能够引导用户准确到达具体目标书架、目标房间。

目标二: 5 秒以内的端-边-云协同响应。将单次定位的端到端延迟控制在 5 秒以内, 支持按需触发而不是持续运行, 避免对移动端造成明显的电池与算力负担。

目标三: 零额外基础设施成本。完全依赖现有 WiFi 覆盖与普通手机摄像头, 方案能够实现在任意已部署 WiFi 的建筑快速复制。

目标四: 以 AR 为主体的直观导航交互。将定位与路径以三维箭头叠加在实景画面上, 消除用户在 2D 地图上的方向感缺失。

目标五: 自然语言意图导航能力。在传统固定 POI 名称输入之外, 支持用户以“我想找个地方讨论”“我想去洗手间”“找一个安静学习的位置”等自然语言表达导航需求, 系统自动解析为最匹配的拓扑目的地节点, 使室内导航从“地点检索”进一步升级为“意图理解”。

1.4 项目挑战

为实现上述目标, 本项目面对以下核心挑战。

挑战一: 多模态的时空一致性。WiFi 属于电磁信号, 精度粗糙但全局鲁棒; 视觉属于光学信号, 精度高但易受纹理干扰。如何设计合理的分层融合策略, 使

两种模态在时空上保持一致，是系统架构上的首要问题。

挑战二：高纹理相似场景下的视觉误匹配。图书馆、医院、商场中存在大量相似的书架、门面、走廊，NetVLAD 全局检索容易得到跨区域的错误候选图像，造成定位漂移。如何在保证检索正确率的前提下压缩检索空间，是精度的关键。

挑战三：端-边-云之间的计算与带宽协同分配。视觉定位完整流水线包含大量卷积与图神经网络推理，在手机上直接跑会发热、耗电且延迟高。如何把重计算卸载到边缘或云端，同时保证端云通信的可靠性，是工程实现的核心。

挑战四：离线建图的人工成本。WiFi 指纹采集和视觉三维重建都需要人工在目标场景中大量采集数据。如何通过采集协议与自动化脚本把建图成本压到“一人一天完成一个楼层”的水平，决定了系统的商业可行性。

挑战五：自然语言意图与空间节点的语义映射。室内导航的真实需求往往不是严格的地点名称，而是带有意图色彩的自然语言表达，例如“找个讨论作业的地方”“去最近的卫生间”。如何在不破坏原有定位与寻路链路的前提下，把用户意图稳定映射到可导航拓扑节点，是系统智能化交互需要解决的新问题。

上述五项挑战共同构成本项目的主要学术与工程难点，后续章节将围绕上述挑战逐一介绍本项目的解决方案。

第二章 设计思路与方案

2.1 技术路线与功能闭环

本节从技术路线与功能闭环角度说明系统如何将用户输入、WiFi 扫描、区域约束、视觉定位、语义拓扑寻路与 AR 呈现串联为一次完整导航服务。本项目面向大型复杂室内环境中的导航需求，设计了一种以 WiFi 指纹网络为基础、以端-云-网协同通信为支撑、以视觉精定位与 AR 导航为增强能力的室内导航解决方案。与传统单一定位方案不同，本系统首先依托建筑内已部署的 WiFi 网络构建低成本空间感知基础，再通过网络传输将终端采集的图像与 WiFi 扫描结果上传至后端，由云端完成高复杂度定位与路径规划计算，最终将导航结果回传至移动端进行 AR 渲染。整个系统形成了“网络侧感知—通信侧协同—云端计算—终端呈现”的完整闭环，体现了网络基础设施、通信链路与智能算法协同工作的技术路线。

从处理流程上看，系统遵循“WiFi 粗定位—区域约束—视觉精定位—多模态语义拓扑寻路与 AR 引导”的分层思路，通过多层次协同逐步提升定位精度、目的地理解能力与导航体验。

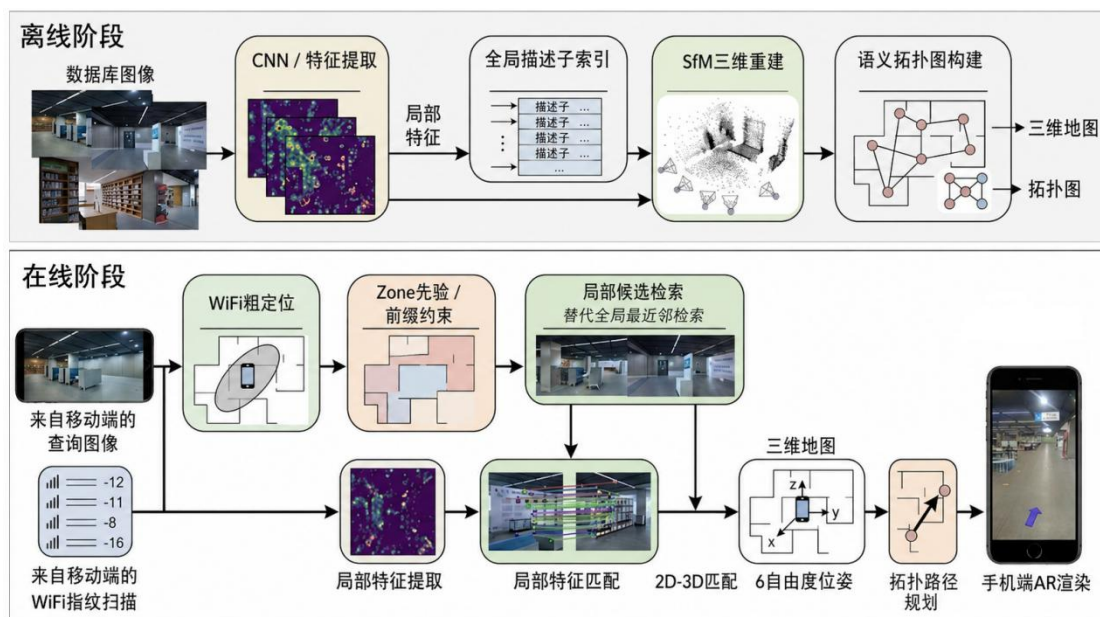


图 1 系统整体流程图

第一步：WiFi 指纹网络感知。系统在离线阶段为每个楼层建立结构化

Radiomap 数据库，将建筑内已有 WiFi 网络转化为可计算、可检索的空间指纹资源。在线阶段，终端通过无线网络扫描获取实时 AP 与 RSSI 信息，并通过网络接口上传至后端；服务端利用 WKNN 算法在较短时间内返回用户所在楼层及二维平面粗坐标。本层的核心意义在于：不依赖额外硬件部署的定位方式，直接将既有 WLAN 通信网络“复用”为一张空间感知网络，为系统提供一个始终可用的全局初始锚点。即使相机尚未参与定位，系统仍能依靠网络侧感知能力判断用户大致所处区域。

第二步：Zone 约束。在获得 WiFi 粗定位结果后，系统并不立即进入全局视觉检索，而是首先利用粗坐标通过空间映射函数将用户划分到预定义的 Zone 区域中。每个 Zone 对应一个语义明确、物理相对独立的子区域，例如某一阅览区、走廊入口或功能空间边界。该区域标签随后作为 Prefix 约束传递给视觉检索模块，限定视觉定位流程中使用的 NetVLAD 模型仅在对应 Zone 的参考图像子集中执行候选检索。这本质上是将 WiFi 网络侧输出的位置先验转化为视觉定位阶段的检索约束，通过网络感知结果直接参与后续智能计算流程。该机制显著压缩了候选搜索空间，降低了计算开销，并有效抑制了跨区域误匹配问题，是系统从“网络定位”走向“多模态协同定位”的关键桥梁。

第三步：高精度视觉精定位。在 Zone 约束确定后，系统对终端上传的查询图像执行高精度视觉定位。查询图像首先通过 SuperPoint 提取关键点与局部描述子，并由 NetVLAD 生成全局描述子；随后在 Zone 约束下完成 Top-K 候选图像检索，并对候选图像使用 LightGlue 进行深度特征匹配，建立 2D-3D 对应关系，最终通过 PnP 与 RANSAC 求解六自由度相机位姿。该层将 WiFi 网络提供的区域级粗定位结果进一步提升为高精度位姿估计，使系统不仅能够判断“用户在哪一片区域”，还能够获得其精确位置与朝向信息，从而为路径引导与 AR 显示提供可靠支撑。

第四层：多模态语义拓扑寻路与 AR 渲染。在获得精确位姿后，系统进一步利用语义拓扑图进行路径规划。具体而言，系统基于 COLMAP 重建的 3D 点云模型对关键空间位置进行人工语义锚定，提炼形成节点数量较少但语义清晰的 JSON 拓扑图，其中节点对应楼梯口、走廊交汇处、区域入口、讨论区、卫生间等关键位置，边权重表示节点间的可达关系与距离。为提升交互智能性，本项目

进一步复用 COLMAP 建图过程中产生的关键帧图像，为每个拓扑节点绑定 `ref_image`、`aliases`、`description` 及语义向量表示，使节点不再只是几何坐标点，而是成为“空间坐标+视觉证据+语义描述”的多模态空间节点。在线阶段，系统可将用户输入的自然语言意图解析为目标节点，再以用户当前位置为起点、目标节点为终点，通过 Dijkstra 算法计算最短路径，并将路径结果通过网络回传至 Unity 前端。前端结合位姿与世界坐标对齐关系，将三维箭头与标签精确叠加到实景画面中的对应物理位置，从而形成“所见即路径”的导航效果。

综上，本项目围绕“网络侧感知、通信侧协同、计算侧增强、终端侧交互”构建了一套面向真实室内场景的完整解决方案。四层模块逐级精化、层层协同：WiFi 指纹网络提供低成本、广覆盖的基础定位能力，Zone 机制实现网络感知结果对视觉检索的前置约束，视觉精定位完成高精度位姿解算，多模态语义拓扑图则把几何路径、关键帧图像与自然语言意图连接起来，最终由 AR 交互将定位结果转化为用户可直接理解的导航指引。通过这种设计，系统在不依赖额外专用硬件的前提下，将网络技术、视觉计算、多模态语义理解与移动交互有机结合，构建了一套符合网络技术赛道定位、且具备实际落地潜力的室内导航解决方案。

第三章 方案实现

3.1 软件架构与模块分层

在上述技术路线基础上，本项目进一步从工程实现角度将系统划分为数据层、算法服务层、业务逻辑层和用户交互层。该分层架构既保证系统内部的模块化与可维护性，也允许 WiFi 定位、视觉定位、语义检索、路径规划和 AR 渲染等能力独立升级。四层自下而上依次为：数据层、算法服务层、业务逻辑层、用户交互层。

数据层负责存储与维护四类核心数据产物：WiFi Radiomap（全楼层指纹库，JSON）、COLMAP 稀疏点云模型（图像、3D 点、特征描述子的.h5 文件集合）、稀疏语义拓扑图（节点-边列表，JSON）、多模态语义索引（节点 ref_image、aliases、description 及其向量表示）。这些数据在离线阶段一次性生成，在线阶段只读，大幅降低运行时的 IO 复杂度。

算法服务层是整个系统的计算中枢，包含四类核心能力：WiFi 定位服务（封装 WKNN）、视觉定位服务（封装流水线，包括 NetVLAD 检索、SuperPoint 特征、LightGlue 匹配、PnP 求解）、语义目的地解析服务（封装 CLIP 图文检索与文本别名匹配）、寻路服务（封装改进的 Dijkstra）。各服务之间通过 Unix Domain Socket 或本地函数调用解耦，延迟低、开销小、部署灵活。

业务逻辑层负责串联各算法服务，完成业务流程编排：接收移动端上报的数据、调度 WiFi → Zone → 视觉的分层定位、将用户输入的固定 POI 名称或自然语言意图解析为 destination_node、调用寻路算法计算路径、把结果打包为统一的响应 JSON 下发至移动端。这一层用 Python 实现，便于快速迭代与调试。

用户交互层由移动端 Unity 应用承担，负责 WiFi 扫描、摄像头取景、AR 渲染、目的地选择、自然语言意图输入、路径展示等直接面向用户的功能。Unity 工程编译为 Android 原生 APK，保证跨机型兼容性。四层之间通过清晰接口解耦，任何一层的升级不会影响其他层，为后续迭代打下工程基础。

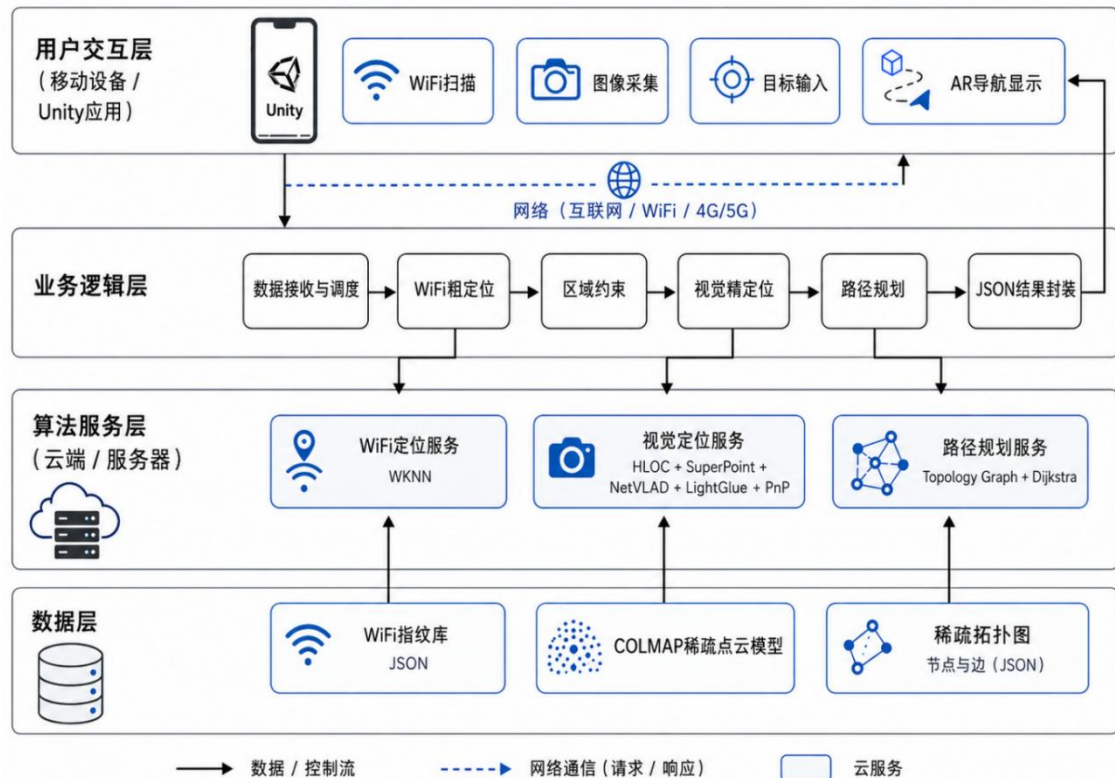


图2 系统实现架构图

3.2 部署架构：“端-云-网”协同

为了在移动端算力受限和视觉算法计算量大之间取得平衡，本项目采用“端-云-网”三方协同的部署架构，把任务合理分配到最适合的节点。

（1）**移动端（Android 手机/平板）**。作为用户直接操作的终端，负责调用系统 WiFiManager 扫描 AP 及 RSSI、调用相机抓一帧 RGB 图像、通过 HTTPS 上传数据至后端、接收返回的路径 JSON 以及调用 Unity 将路径、箭头、目的地标签渲染为 AR 图层叠加在预览画面上。移动端不做任何深度学习推理，也不做复杂几何计算，最大的计算开销仅是渲染，因此任何能跑 Unity AR 应用的手机都可以作为终端，硬件门槛很低。

（2）**网络层**。采用 HTTPS 协议与后端通信。每次定位请求只需上传一张压缩后的 JPEG（通常小于 200KB）和一段 WiFi 扫描结果 JSON（通常不足 10KB），带宽压力极小；响应回传的路径 JSON 更轻（通常不足 5KB），弱网下也能稳定工作。对于医院、政府等注重隐私的场景，网络层也支持部署在局域网内部，数据不出场区，满足合规要求。

(3) **后端层（边缘或云端服务器）**。后端是算力核心，承载 WiFi 定位、视觉定位、路径规划等全部重计算任务。硬件配置可以按场景规模伸缩：单建筑（如一座图书馆）用一台配消费级 GPU（如 RTX 3060）的边缘盒子即可支持数百并发；连锁场景（全国商场）则可部署到公有云 GPU 集群，通过负载均衡分发请求。软件栈包括 Python 3.10+、PyTorch 2.x、COLMAP、Hloc、Flask/FastAPI 等。我们特别采用 Unix Domain Socket 作为视觉定位常驻服务与业务进程之间的通信桥梁，相较传统 TCP 可将单次调用开销降低一个数量级。

(4) **数据层**。可以物理集成于后端服务器，也可独立部署于对象存储或文件系统。核心数据包括 Radiomap JSON、COLMAP 模型（GB 数量级，取决于场景规模）、SuperPoint/NetVLAD/LightGlue 预训练权重（共数百 MB）、拓扑图 JSON。所有数据在建图生成后冻结只读，运行时无写入操作，便于以 CDN 或对象存储方式分发更新。

综上，本系统将“采集与渲染”留在端侧、“定位与规划”集中于后端，以极低的硬件门槛和带宽开销，实现从数据采集到 AR 导航的完整闭环。

3.3 链路运转流程

下面对一次完整的定位-导航请求做逐步拆解，以便理解系统在运行时各模块之间的数据流与时序。

步骤 1：用户触发。用户打开 APP，输入目的地（如“卫生间”）后点击“开始导航”。

步骤 2：终端数据采集。APP 同时发起两个异步任务：

(a) 调用 `WiFiManager.getScanResults()` 获取当前可见 AP 列表及 RSSI。

(b) 调用 `Camera2` 捕获一帧高分辨率 RGB 图像并压缩为 JPEG。两个任务通常在 200 毫秒内完成，几乎无延迟感知。

步骤 3：数据上传。APP 把 WiFi 扫描结果打包为 JSON，图像放在 HTTPS 请求体中，向后端接口发 POST 请求。

步骤 4：WiFi 粗定位。后端 Python 业务进程调用 `WiFiLocator.get_location()`，在 Radiomap 上跑 WKNN ($K=3$)，返回三维粗坐标($x, y, floor$)，耗时约 10~30 毫秒。

步骤 5: Zone 映射。业务进程调用区域配对函数，在得到粗坐标(x, y)之后，根据预先硬编码的 Zone 边界规则判断用户所在 Zone，返回字符串列表（如 [“C”, “D”]），耗时微秒级。

步骤 6: 视觉定位。业务进程通过 Unix Domain Socket 将图像文件名与 Zone 列表发给常驻的视觉精定位模块。视觉精定位模块用 SuperPoint 提取局部特征、NetVLAD 计算全局描述子、在 Zone 约束下的候选库中做 Top-K (K=10) 检索，对每对候选图像用 LightGlue 匹配，最后用 PnP+RANSAC 求解位姿(R, t)，耗时约 3~5 秒。

步骤 7: 坐标系转换。视觉精定位模块返回的是 COLMAP 相机坐标系下的位姿，需通过 $C_{world} = -R^T t$ 转到世界坐标系下，然后映射为数据预处理阶段采集的真实世界坐标。业务进程完成，耗时毫秒级。

步骤 8: 路径规划。业务进程调用寻路模块，以用户当前位置为起点、用户指定的目的地节点为终点，在稀疏拓扑图上使用 Dijkstra 算法，返回关键节点三维坐标数组。耗时毫秒级。

步骤 9: 结果下发。业务进程将路径 JSON（起点、节点序列、终点、总距离）作为 HTTPS 响应返回 APP。

步骤 10: AR 渲染。APP 的 Unity 引擎收到路径 JSON，通过 Inverse 矩阵把每个世界坐标转换为 Unity 本地坐标，在 AR 画面中对应的物理位置实例化三维箭头预制件，为用户指明方向。

从用户点击到渲染出 AR 箭头的端到端全链路流程延迟约 4~5 秒，绝大部分时间消耗在视觉定位的神经网络推理环节。后续的优化空间主要集中在这一环节的进一步加速（模型量化、ONNX/TensorRT 转译等）。

3.4 终端用户使用流程

从终端用户视角看，系统的使用流程可以用四个步骤描述。

第一步：启动 APP。用户进入建筑后打开 APP，系统进入 AR 导航准备界面，并提示用户面向走廊、入口或具有明显视觉特征的区域，以保证后续视觉定位具有足够的图像纹理和空间辨识度。

第二步：输入目的地。用户可以在输入框中输入标准 POI 名称，也可以输

入自然语言意图，例如“我想找个地方讨论作业”“我要去卫生间”“找一个安静学习的位置”。移动端将目的地文本、WiFi 扫描结果与当前图像一并上传至后端，后端通过名称匹配或多模态语义检索将用户输入转换为拓扑图中的目标节点 ID。

第三步：定位与路径规划。用户点击“开始导航”后，移动端完成 WiFi 扫描和图像采集，并通过网络接口提交至后端。后端首先利用 WiFi 指纹结果获得区域级粗定位与 Zone 先验，再在 Zone 约束下调用 Hloc 视觉定位流程求解用户六自由度位姿，随后以当前位姿为起点、目标节点为终点，在语义拓扑图上执行 Dijkstra 路径规划，并将路径坐标序列返回至移动端。

第四步：AR 箭头引导。Unity 前端根据后端返回的位姿与路径坐标，完成 COLMAP 坐标系与 Unity AR 坐标系的对齐，并在摄像头实景画面中渲染三维箭头。用户跟随箭头方向前进，系统根据用户与当前路径节点的距离自动切换下一个目标节点，直至到达目的地。

3.5 基于指纹库的 WiFi 粗定位模块

WiFi 指纹定位是本系统定位流水线的第一级，其作用是在用户任意位置、任意朝向下，快速给出一个米级精度的三维坐标估计（平面物理坐标和所在楼层），为后续视觉定位提供 Zone 级语义先验。本模块基于国际室内定位领域最经典的开源系统之一——由塞浦路斯大学 DMSL 实验室开发的 Anyplace。Anyplace 以 WiFi 指纹+Radiomap+KNN 为核心架构，曾在 2014 年微软室内定位大赛中取得 1.96 米定位精度的第二名，是 WiFi 指纹定位的代表性工程实现。

在充分吸收 Anyplace 思想的基础上，本项目针对“单机部署、Python 后端”的实际需求做了两方面重构：其一，我们提取了 Anyplace Logger 的核心算法，合成了一个轻量化的新应用用于离线数据库的采集与真实物理坐标的标定；其二，我们首先测试了 Anyplace 官方 Android Navigator 在线定位的可靠性，确认算法正确后将其核心逻辑从 Java/Kotlin 移植到 Python，封装为 WiFiLocator 类作为后端的标准调用接口。整个 WiFi 模块分为离线采集与在线匹配两部分，下面两小节分别展开说明。

3.5.1 离线采集端重构：Anyplace Logger 采集 WiFi 指纹数据与建立 Radiomap

离线采集的核心任务是在目标楼层的每一个参考点采集其所能接收到的所

有 WiFi AP 的 RSS 值，形成一张以位置为索引、以 AP 列表为列、以 RSS 值为单元格的三维指纹矩阵——Radiomap。这一阶段的数据质量直接决定了在线匹配精度的上限。

(1) Logger 应用的裁剪与合成。Anyplace 开源项目自带一款名为 Anyplace Logger 的 Android 应用，支持用户在建筑楼层平面图上点击参考点、按下“开始采集”按钮后连续记录 WiFi 扫描日志。原版 Logger 依赖 Anyplace 云后端（用于同步楼层图与上传日志），代码体量庞大。我们对 Logger 源码做了深度剥离，保留其核心工作流，去除云后端同步模块，替换为本地文件系统存储，UI 层简化为“设置比例尺—采样记录—导出日志”三个步骤。合成后的 Logger 单 APK 即可运行，不需要任何云端配置，大幅降低了采集工作的门槛。

(2) 采集策略与坐标标定。采集遵循 Anyplace 官方推荐的“Point-by-Point”方法：在目标楼层选择一系列相距 2~3 米的锚点，每个锚点以 4 个方向（东、南、西、北）各采集多个 WiFi 扫描样本，通过方向平均降低人体遮挡与天线极化带来的系统性误差。以图书馆五楼楼层为例，我们共采集了约 320 个锚点，每个锚点每个方向 3~4 次，累计约 7000 条原始 RSS 扫描记录，每条记录包含锚点坐标 (x, y) 以及该次扫描中所有 AP 的 BSSID 与 RSSI。为了让输出坐标与物理空间的米数精确对齐，我们使用 Logger 的“比例尺标定”功能，通过在底图上点击两个实际距离已知的锚点来校准图上像素与物理米数之间的映射关系。

(3) Radiomap 数据清洗与结构化。原始日志是按扫描时间排序的文本，直接使用会因噪声与异常值导致精度下降。我们编写了一段 Python 脚本完成清洗：剔除短时间内 RSS 剧烈抖动（超过 $\pm 10\text{dBm}$ ）的异常 AP；对同一锚点、同一方向、同一 AP 的多次 RSS 样本取均值；对任一锚点未检测到的 AP，填充默认值 -110dBm （低于大多数手机 WiFi 模组的灵敏度下限，表示“不可见”）。清洗后汇总为一张统一的 JSON 结构化 Radiomap，包含“all_bssids”（全场景 AP 的 BSSID 列表）和“floors”（按楼层组织的参考点列表，每个参考点含坐标与指纹向量）两个顶层字段。这一结构既便于 Python 后端高效加载，也天然支持按楼层、按区域灵活检索。

(4) 数据规模与质量控制。以学校图书馆为例，每楼层的 Radiomap 有约 320 个有效参考点、78 个可见 AP、约 25000 个非默认 RSS 条目，总体积约 1.8MB。

本项目采用留一法交叉验证评估质量：在 $K=3$ 配置下平均定位误差 1.83 米，75%分位数误差 2.54 米，与 Anyplace 官方在真实建筑中的精度水平一致。



图 3 Radiomap 构建结果示例

3.5.2 在线匹配算法实现

在线匹配是 WiFi 定位的“使用”阶段，目标是把用户实时上传的 WiFi 扫描结果与离线 Radiomap 做比对，快速计算三维坐标估计。项目首先测试了 Anyplace 官方 Android 版的在线定位的可靠性，接着将其核心 WKNN 算法剥离并重构为 Python 语言，封装为 WiFiLocator 类，作为整个导航系统后端的标准调用接口。整体在线匹配流程为：实时 WiFi 扫描 → 生成实时指纹向量 → 与离线指纹库计算相似度 → WKNN 算法计算位置 → 多源融合优化。

(1) **实时指纹向量生成。**用户手机上报的 WiFi 扫描结果是一组“BSSID → RSSI”键值对，通常包含 10~30 个可见 AP。模块把它解析后，在离线 Radiomap 的 all_bssids 列表中按顺序对齐：对于当前可见的 AP 使用实时 RSSI，对于不可见的 AP 填默认值 -110 dBm，该值与离线阶段一致，从而在信号空间中构造同维度的指纹向量。输出的指纹向量维度为 |all_bssids|，本项目中约为 78 维。

(2) **信号空间欧氏距离计算。**WKNN 的基本假设是：两个物理位置越近，其 RSS 指纹向量在信号空间中也越相似。基于此，我们将用户实时指纹向量与 Radiomap 中每一个参考点的指纹向量计算欧氏距离。给定用户向量 $s = [s_1, \dots,$

$s_M]$ 与参考点向量 $r = [r_1, \dots, r_M]$, 其距离定义为 $d(s, r) = \sqrt{(\sum_i (s_i - r_i)^2)}$ 。距离越小, 表示用户越可能位于该参考点附近。

(3) **Top-K 最近邻选取**。将所有参考点按距离升序排序, 选前 K 个作为候选邻居。 K 的选择是一个经典权衡: K 太小结果对单点噪声敏感, K 太大则引入远距离干扰邻居降低精度。按 Anyplace 默认配置以及我们的实验测试, 默认取 $K=3$, 并在实现中支持 $[1, 10]$ 区间动态调整。

(4) **加权位置估计**。与传统 KNN (简单平均邻居坐标) 不同, WKNN 把“距离越近权重越大”的直觉显式建模, 用欧氏距离倒数作为权重。第 i 个最近邻居的权重为 $w_i = 1/(d_i + \epsilon)$, ϵ 为防止除零的小量 (实现取 $1e-6$)。用户位置估计为:

$$\hat{x} = (\sum_i w_i \cdot x_i) / (\sum_i w_i), \hat{y} = (\sum_i w_i \cdot y_i) / (\sum_i w_i)$$

其中 (x_i, y_i) 是第 i 个最近邻居的物理坐标。这种加权方式能在信号空间的局部线性邻域内做平滑插值。文献普遍表明 WKNN 相对纯 KNN 可获得约 20~30% 的精度提升。

(5) **异常保护**。实现中额外加入两项优化以提升鲁棒性。异常值过滤: 当用户扫描结果中可见 AP 少于 3 个 (处于盲区或 WiFi 刚开启) 时, 直接返回 None 并降级, 避免信息不足时给出不可靠估计; BSSID 清洗: 构造扫描字典时对 BSSID 做 strip 去空白与小写规范化, 消除因不同厂商 WiFi 驱动在 MAC 格式化风格上的差异, 从而避免匹配失败。

(6) **性能实测**。在学校图书馆场景下, 单次 WKNN 在线匹配的端到端耗时 (不含网络开销) 约 15~25 毫秒, CPU 占用可忽略。其输出的三维坐标将用于 Zone 映射, 并接入后续视觉定位流水线。综合来看, 本模块在精度、延迟、鲁棒性三方面均达到实际部署水平。

3.6 高精度分层视觉定位模块

视觉定位模块是本系统实现亚米级定位精度的核心, 其实现基于 ETH Zurich 计算机视觉组开源的 Hierarchical Localization (Hloc) 框架。Hloc 是当前视觉定位领域最具代表性的开源工具箱之一, 曾于 CVPR 2020 与 ECCV 2020

的室内外视觉定位挑战赛中双料夺冠，其核心流水线包括：NetVLAD 全局描述子检索、SuperPoint/DISK 等深度局部特征提取、SuperGlue/LightGlue 特征匹配、PnP+RANSAC 位姿求解四个关键环节。

我们将 Hloc 作为 Python 库集成至视觉高精度定位模块中，流程基本遵循 Hloc 的标准范式。针对本项目的实际需求做了三处工程化适配：一是提供一套适合室内大规模重建的离线建图脚本（视频抽帧、人工关键配对、SuperPoint+SuperGlue 重建）；二是把 Hloc 的在线定位接口封装为常驻服务（通过 Unix Domain Socket 通信），避免每次定位都承担模型加载的高昂开销；三是在 Hloc 原生检索的基础上加入 Zone Prefix 约束，把检索空间从全库压缩到 Zone 子集。下面分两个小节介绍离线重建与在线定位的实现细节。

3.6.1 离线稠密重建：分区视频流采样与三维点云生成

离线重建阶段的目标是为目标场景建立一个精确的、可供在线查询使用的三维点云模型。本项目以学校图书馆四楼与五楼为验证场景，建立覆盖约 2400 平方米连通空间的 COLMAP 模型，总计约 46 万个三维特征点。整个重建流程的每一个环节——从视频采集、抽帧、人工关键配对到 COLMAP 全局捆绑调整——都对最终精度至关重要。

（1）视频采集策略。为了同时兼顾覆盖与闭环约束，我们设计了多层次采集方案：主要走廊按“正向-反向”两次分别录制形成双向覆盖；每个独立子区域（人文社科阅览区、自然科学阅览区、自习区、期刊阅览区等）做内部闭环录制；最后对整体地图做一轮总闭环录制，把各子区域在视觉特征空间中粘合起来。视频录制采用 4K-30fps 规格，以正常步速缓慢平移以避免运动模糊，相机保持水平朝向符合实际手持姿态。

（2）图像抽帧与预处理。我们以每秒 1 帧的频率均匀抽帧，得到覆盖全场景的约 9600 张静态图像。抽帧率的选择是精度与计算开销的权衡——更高的抽帧率带来更丰富的多视角覆盖，但也拖慢了 COLMAP，1Hz 经实验证明是合理的折衷。抽帧后图像按区域分文件夹组织，统一做尺寸压缩（长边限制为 1920 像素），以匹配消费级 GPU 的计算规模。

（3）图像配对的人工关键干预。这是本项目在 Hloc 官方流水线之上最重要的一项工程改进。Hloc 默认用 NetVLAD 对全库图像两两做特征相似度排序，

对每张图取 Top-K 相似邻居作为 SfM 候选配对。但在图书馆这类高纹理相似场景中，NetVLAD 极易把不同物理位置但纹理相似的图像错误配对，进而在 COLMAP 重建中触发灾难性漂移（把物理上相距几十米的位置错误合并，形成扭曲地图）。为此我们没有使用 NetVLAD 配对，而是采取两步策略：其一，序列配对（pairs_from_sequence），利用视频帧的时序连续性为每一帧生成与相邻 N 帧的配对，其中 N 依据每个区域的共视连续性质量决定，这种配对在空间上绝对可靠；其二，人工拼接配对，在不同子区域之间标注数十对关键配对（走廊两端的视角、区域交汇处的对接图像），为 COLMAP 提供可信的闭环信号。标注图书馆一层楼约耗费 2 小时，这一干预工作确保了重建模型在物理空间中一致。

（4）特征提取与匹配。获得可靠配对后，我们对每张图用 SuperPoint 深度网络提取关键点与 256 维描述子。SuperPoint 由 Magic Leap 于 2018 年提出，相比传统 SIFT/ORB，在光照变化、运动模糊等条件下鲁棒性更强，是 Hloc 默认首选。对每对配对图像，我们用 SuperGlue 做匹配。SuperGlue 由 ETH Zurich 于 CVPR 2020 提出，基于图神经网络、通过自注意力与交叉注意力对两图特征点做全局关系建模，匹配正确率相较传统“最近邻+比率检验”可提升 30%以上。SuperGlue 推理速度虽慢于后继 LightGlue，但匹配精度更高，是离线阶段的最优选择。

（5）COLMAP 三维重建。获得全部图像的特征与成对匹配后，我们将其导入 COLMAP 做全局 SfM 重建。COLMAP 是开源三维重建的事实标准，包含增量式图像注册、三角化、捆绑调整等关键环节。针对本项目，我们做了几项参数调优：采用单目相机模型（SIMPLE_RADIAL）并预先标定内参，避免重建中反复优化内参导致的过拟合；图像压缩为长边 1920 像素；把匹配阈值 min_num_matches 设为 15 以剔除低质量图像对；启用全局捆绑调整并设置较严格的收敛阈值。最终模型包含约 46 万稀疏三维点、约 9600 张图像的精确位姿，以及可供 Hloc 在线查询的 .h5 特征索引文件，总体积约 3.2GB。

（6）模型验证。重建完成后我们随机选若干已知物理位置的查询图像通过 Hloc 接口做验证。实测结果显示，模型在图书馆 4、5 楼全场景下的平均重投影误差小于 1 像素，查询图像定位误差平均小于 0.6 米，满足亚米级精度要求。



图 4 楼层点云模型示例

3.6.2 快速精确定位

在线定位阶段以前述离线重建得到的 COLMAP 模型为地图，以用户查询图像与 Zone 约束为输入，输出查询图像对应的 6 自由度相机位姿。本模块是整个系统实时链路中最耗时的环节，性能直接决定用户等待时长。为此，项目在“检索速度”与“匹配精度”两条路线上分别使用最适合的模型。

（1）**NetVLAD 全局特征预计算与 Top-K 检索。**离线建图的尾阶段，我们为每张库图像预先计算并持久化保存其 NetVLAD 全局特征向量（4096 维），存于 HDF5 文件，便于在线阶段高速加载。查询图像到达时，系统先用同一 NetVLAD 网络对查询图做前向推理得到 4096 维全局描述子，然后在 Zone Prefix 约束下的候选子集（而非全库）中做余弦相似度计算，选 Top-K（K=10）作为候选库图。得益于 Zone 约束，候选集规模从全部范围的约 9600 张压缩到单 Zone 平均约 100 张，检索耗时从全库的约 300 毫秒降到约 30 毫秒，降低一个数量级。

（2）**查询图像的 SuperPoint 特征提取。**我们用与离线阶段相同的 SuperPoint 模型对用户上传的查询图像提取关键点与 256 维局部描述子。为保证一致性，我们将关键点数量上限、NMS 阈值等参数设置为与离线阶段完全相同。特征提取在 GPU 上约 60~100 毫秒。

（3）**基于 LightGlue 的快速特征匹配。**与离线阶段采用的 SuperGlue 不同，在线阶段我们采用其后继者 LightGlue 作为特征匹配器。LightGlue 由 ETH

Zurich 于 2023 年提出，在保留 SuperGlue 自注意力+交叉注意力匹配思想的同时，引入自适应深度、点剪枝、早期终止三项关键机制，能根据输入图像对的难度动态调整计算深度，在“容易配对”的情形下提前结束推理，从而大幅加速。实测显示 LightGlue 在保持与 SuperGlue 相当（甚至略优）的匹配精度的前提下，单次推理平均降低一半以上：10 对候选图像的批量匹配，从 SuperGlue 的约 2.5 秒降到 LightGlue 的约 1.2 秒。LightGlue 的引入是本项目端到端延迟能控制在 5 秒以内的关键优化之一。

（4）PnP+RANSAC 鲁棒位姿求解。对每对“查询图—候选库图”完成特征匹配后，通过库图的 2D 关键点到 COLMAP 三维点的已知对应关系，构造“查询图 2D 点 → COLMAP 3D 点”的对应集合。Hloc 把所有候选库图的对应集合汇总后，通过 pycolmap 封装的 PnP 算法配合 RANSAC 鲁棒估计，求解满足最大内点一致性的相机位姿 (R, t) ，即 3×3 旋转矩阵与 3×1 平移向量。RANSAC 可有效过滤 5%~10% 的误匹配外点，保证在噪声存在时位姿求解依然稳定。这一步在 CPU 上约 100~200 毫秒。

（5）相机中心位姿的世界坐标系变换。PnP 返回的 (R, t) 是“世界点到相机坐标系下的变换”，其中 t 并不是相机在世界坐标系中的位置，而是世界原点在相机坐标系下的位置——这是 COLMAP 约定的标准但容易引起误解的惯例。根据射影几何，相机中心在世界坐标系中的位置为：

$$C_{world} = -R^T t$$

即对旋转矩阵做转置（旋转矩阵是正交矩阵，转置即其逆），再与平移向量的负值相乘。这一步是后续 Dijkstra 寻路、AR 锚点对齐等模块正确运行的前提。我们把这一变换写成独立的、经过严格单元测试的坐标转换函数，并在日志中打印中间结果，以便出问题能够快速诊断。

（6）与 Zone 先验的一致性校验。位姿求解完成后，我们把位姿投影回二维楼层坐标，与 WiFi 定位给出的粗坐标做一致性校验。若两者二维距离超过 5 米（显著大于正常融合误差），我们认为这次视觉定位可能意外漂移，触发降级策略——退回 WiFi 粗坐标，并在 APP 端提示“请换一个位置再试”。这一机制简单但有效，能在极端情况下守住体验的下限。

3.7 Prefix 约束式空间筛选机制

Prefix 约束式空间筛选是本项目三大核心创新之一，也是 WiFi 定位与视觉定位之间的“语义桥梁”。其核心思想是把 WiFi 定位结果不再视为一个独立的位置估计，而是视为一个语义先验，用于在视觉流水线真正启动之前就对其检索空间做显式压缩。

3.7.1 Zone 区域先验与检索空间降维

我们将待检索的区域根据 x, y 坐标划分为 32 个 Zone 区间，在线时根据 WiFi 定位返回的二维坐标，通过预设的映射规则（if-else 逻辑）确定 Zone 归属。下面展开说明设计与实现细节。

（1）Zone 的设计原则与划分方法。我们将图书馆 4、5 楼约 2400 平方米可达区域划分为 32 个互不重叠、覆盖全空间的 Zone。划分遵循三个原则：Zone 内部纹理相对自成一体（进入一个 Zone 后所见的视觉特征彼此相似度高于跨 Zone）；Zone 尺寸通常在 3~8 米量级，与 WiFi 米级精度相匹配；Zone 边界尽量与物理隔断（墙、货架、楼梯）对齐，便于直观定义与标注。32 个 Zone 的标注约耗时两个工作日，以一组二维矩形或不规则多边形边界坐标存储。

（2）Zone 映射函数 `locate_regions`。我们把 Zone 边界硬编码为一条简洁的 if-else 规则链，输入为 WiFi 坐标，输出为所落入的一个或多个 Zone 名称。之所以允许多个 Zone，是因为 WiFi 定位本身有米级误差，当坐标落在边界附近时能够同时返回两个相邻 Zone，降低因定位不准而错过正确 Zone 的概率。实测在 32 个 Zone 的划分下，边界附近的双 Zone 返回比例约 12%，其余 88% 返回单一 Zone。

（3）视觉检索空间的实际压缩效果。无 Zone 约束时，NetVLAD 的候选库是全楼层约 9600 张图像；引入 Zone 约束后仅在当前 Zone（或双 Zone）对应的约 100~200 张图像中检索，压缩比约 95~98%，检索耗时从约 300 毫秒降到 30 毫秒以内，降幅约 10 倍。更关键的是跨 Zone 误匹配从源头被消除——即使某张库图在 NetVLAD 特征空间中与查询图极度相似，只要不在当前 Zone 候选集中就不会被召回。这一机制在高纹理相似场景中至关重要。

（4）与 LightGlue 的协同。Zone 约束不仅压缩了检索空间，还连带减少 LightGlue 的匹配次数（从全库 Top-K 匹配降为 Zone 内 Top-K 匹配）。结合 LightGlue 本身的自适应早停机制，单次视觉定位的端到端延迟从无约束时的

12~15 秒显著压缩到 4~5 秒，这正是本项目实测延迟能够稳定在 5 秒以内的关键原因。

(5) 与降级策略的兼容。当 WiFi 定位完全失败（例如用户处于盲区、Radiomap 未覆盖的角落）时，locate_regions 会返回一个默认的“G”（Global）Prefix，视觉流水线退回全库检索，保证极端情况下的可用性。这种“默认降级”的设计让 Zone 约束成为一种“锦上添花”的优化，既发挥其性能优势又不牺牲最小可用性。

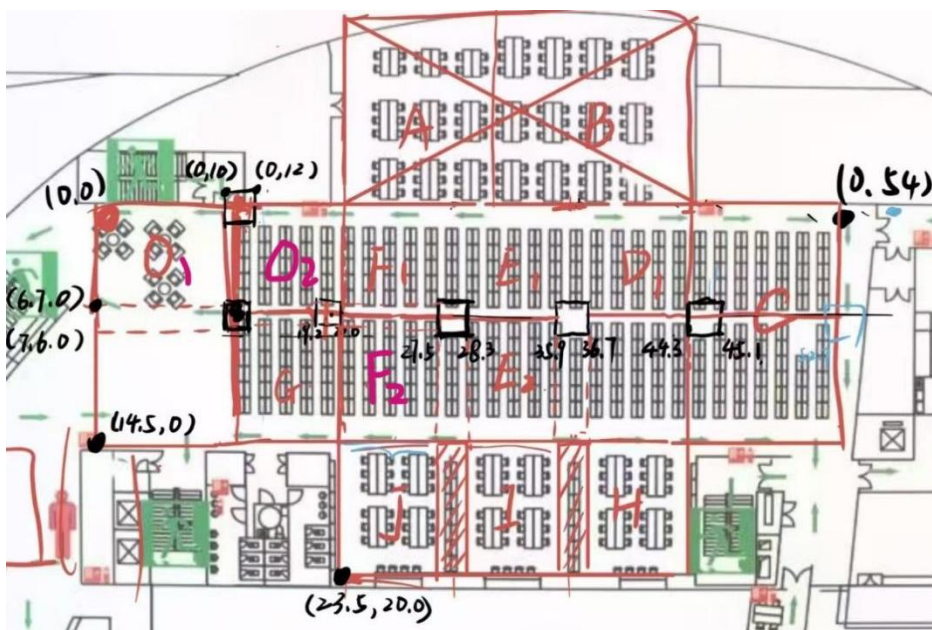


图 5 Zone 区域划分结果示例

3.8 基于稀疏语义拓扑图的寻路算法

视觉定位模块输出精确的 6 自由度位姿之后，导航任务并未结束——用户真正需要的是“从当前位置到目标位置该怎么走”，这本质上是一个路径规划问题。为此我们设计了基于稀疏语义拓扑图的寻路子系统，以解决“海量点云中直接寻路开销巨大”和“纯几何路径缺乏语义可编辑性”两大痛点。本节分两个小节分别介绍语义锚定与寻路算法。

3.8.1 物理轨迹提纯：从稠密点云到稀疏拓扑的语义映射

COLMAP 重建得到的原始模型在图书馆场景下包含约 46 万个稀疏三维点，每个点都是由多幅图像三角化得到的场景角点、边缘点或纹理点。若直接在这 46 万点上构造 A* 或 RRT 寻路图，会面临三个问题：其一，空间复杂度过高（距离矩阵规模达 529 亿量级），无法在毫秒内求解最短路径；其二，点云大多

是“视觉特征点”，并不对应真实可行走空间；其三，缺乏语义，求出的路径虽然数学上最短，却可能穿墙跨柜。

为此我们设计“人工肉眼语义锚定”的方法，把各楼层 COLMAP 点云压缩为一张统一的跨楼层稀疏拓扑图。具体流程如下。

第一步：逐楼层语义锚定。对建筑内每一个楼层，分别在 COLMAP 点云可视化界面中打开对应楼层的重建模型，用三维鼠标自由导航。点击点云中任意一点，COLMAP 会显示该点的 3D 坐标及其关联的所有图像视角。我们通过查看关联图像判断该点的物理位置（如“东楼梯口”“B17 区入口”），完成一次“视觉—语义”人工锚定。每个节点在记录三维坐标的同时附加 floor_id 字段，标明其所属楼层，为后续跨楼层图的统一建模提供层级索引。

第二步：逐楼层节点选取、关键帧绑定与跨层锚点对标注。在每一楼层按导航需求选取关键位置作为拓扑节点，涵盖三类：结构性关键点（门、楼梯、走廊交汇点，约 50%）、各区域入口（约 30%）、重要 POI（服务台、自习区、打印区等，约 20%）。每个节点记录真实三维坐标、所属 Zone、自然语言 label 以及来自 COLMAP 建图视频帧的 ref_image。对于楼梯口、电梯口等垂直通道，需在相邻两个楼层各标注一个对应节点，构成“跨层锚点对”——例如三楼楼梯口节点与四楼楼梯口节点，二者在物理空间上对应同一部楼梯的上下两端，是后续跨楼层连边的基础。

第三步：同层连边与跨层连边的分类标注。节点选取完成后，人工标注节点间的可达关系形成无向边，分两类处理。同层边：若两节点位于同一楼层且物理空间上存在无遮挡可行走路径（不穿墙、不穿柜）则添加一条普通无向边，权重按两端点水平欧氏距离自动计算。跨层边：每对“跨层锚点对”之间添加一条跨楼层无向边，权重在水平距离基础上叠加可配置的楼层切换惩罚系数（默认等效于额外步行 10 米），并在边属性中标注通道类型（stairs/elevator）与方向（up/down），供前端 AR 渲染时展示对应的楼层切换提示。单一楼层中等复杂度约产生 80~120 条同层边；整栋建筑的跨层边数量通常与垂直通道数量相当，额外开销极小。全部节点与边以统一 JSON 文件持久化，总体积通常不超过 50KB。

第四步：迭代验证，重点覆盖跨楼层路径。完成初版后，在 APP 中触发典型导航需求，验证分两个层次进行：单层路径验证（如 A 区到卫生间、走廊交

汇点到阅览台），检查路径是否符合常识性直觉；跨层路径验证（如一楼入口到三楼某阅览区），重点检查 Dijkstra 是否正确选择了楼梯或电梯、楼层切换提示是否在正确位置触发、路径在楼层交接处是否连续自然。若路径不合理则调整节点坐标、边连通关系或楼层切换惩罚系数，通常 2~3 轮迭代后收敛。

这种“人工肉眼锚定”初看似乎不够自动化，但工程价值在于：一次性投入约 4~6 小时标注，可换来一张语义清晰、开销极小、可编辑性强的拓扑图；场景变化（临时施工、新增书架）时也可通过简单编辑 JSON 快速反映到导航结果，对商业化运营至关重要。更重要的是，节点绑定的 `ref_image` 并非额外采集成本，而是对 COLMAP 关键帧资产的再利用，使后续多模态语义检索可以直接建立在已有建图数据之上。最终在图书馆 4、5 楼场景下构建出一张包含 65 个节点、96 条无向边的稀疏拓扑图，相对于 46 万点稠密点云实现了约 5000 倍的数据降维。



图 6 拓扑图构建结果示例

在数据结构上，传统拓扑节点通常只包含(x, y, z)坐标与邻接边。本项目将其扩展为(x, y, z, `ref_image`, `aliases`, `description`, `embedding`)的多模态节点表示。例如 Discussion Area 2 节点不仅保存 COLMAP 世界坐标，还保存该位置对应的关键帧图像、中文别名集合（如“讨论区”“会议区”“交流区”）以及“适合多人讨论、开会和交流的空间”等语义描述。这一设计使拓扑图同时具备几何可达性、视觉可解释性和语言可检索性。

3.8.2 在线图论求解：基于物理绝对坐标的跨楼层 Dijkstra 路径规划

拓扑图构建完成后，在线寻路的核心任务是：给定用户当前位姿（由视觉定位模块提供）与用户指定或语义解析得到的目标节点，在跨楼层拓扑图上求出一

条最短可行路径。算法流程与工程实现细节如下。

(1) **跨楼层图结构建模**。系统启动时解析 JSON 格式的先验拓扑文件，该文件统一描述建筑内所有楼层的节点与边，形成一张全局无向图。我们采用字典+邻接表表示：以节点 ID 为键、以邻居列表为值，每个邻居记录其 ID 与边权重。同楼层节点间的边权重按两端点水平欧氏距离计算，直接反映步行代价；跨楼层边权重则在水平距离基础上叠加一个可配置的楼层切换惩罚系数（默认等效于额外步行 10 米）。

(2) **起点锚定：跨楼层感知的最近节点投影**。用户位姿 C_world 是一个连续的三维坐标，系统在做最近节点投影时首先以 WiFi 定位模块返回的 floor 字段过滤，仅在当前楼层的节点集合中寻找最近有效节点作为“虚拟起点”，避免三维空间中上下楼层节点在竖直方向上距离相近而导致的错误锚定。实测下用户位姿与其最近同层拓扑节点的距离通常小于 3 米，由此引入的起点误差对整体路径长度影响可忽略。

(3) **Dijkstra 最短路径求解**。以锚定得到的“虚拟起点”为源节点、用户指定或自然语言解析得到的目标节点为终节点，调用 Dijkstra 算法在全局跨楼层拓扑图上执行路径求解。Dijkstra 是经典的单源最短路径算法，由荷兰科学家 E. W. Dijkstra 于 1956 年提出，至今仍是非负权图最短路径问题的首选解法。其基本思想是维护“已知最短距离”集合与“候选距离”优先队列，每次从队列取出距离最小的节点做松弛操作，直到到达目标节点。对于本项目规模的多楼层拓扑图，使用二叉堆优化的 Dijkstra 时间复杂度为 $O((n+m) \log n)$ ，现代 CPU 上单次求解不足 1 毫秒，扩展到全楼层后计算开销依然可忽略。

(4) **路径的楼层分段提取与视觉定位模型切换**。Dijkstra 返回节点 ID 序列后，系统按 floor_id 对路径做分段切割，输出一个有序的“楼层段列表”，每段包含该段所属楼层及其对应的三维坐标子序列。这一分段结构驱动两个下游动作：其一，前端 AR 渲染按楼层段分批绘制路径箭头，当用户到达跨层锚点（楼梯口/电梯口）时自动切换至下一楼层段的引导；其二，后端视觉定位模块持续监听 WiFi 定位模块返回的 floor 字段，一旦检测到楼层号发生变化，立即切换至对应楼层的 COLMAP 模型，确保在线查询始终在正确楼层的点云中检索。这一机制对用户完全透明——无需任何手动确认，系统在用户踏上新楼层的第一次 WiFi

扫描周期内即完成模型切换，视觉定位无缝衔接。

(5) **起点坐标注入：消除前端渲染的视觉断层。**若直接将 Dijkstra 返回的节点路径交给前端渲染，AR 画面中的第一个箭头将出现在“虚拟起点”（即最近的拓扑节点）处，而不是用户实际所在位置。两者之间可能有 1~3 米距离，会让用户感受到“路径从前方几米远的地方才开始”，产生明显的不协调感。为此，模块在路径序列的最前端强制注入用户的真实初始位姿 C_world ，使前端渲染时从用户脚下自然“生长”出路径，消除视觉断层。

(6) **封装与输出。**系统把完整路径封装为 JSON 响应体，结构为：首端注入的用户真实位姿、各楼层段的三维坐标子序列（附 $floor_id$ 标识）、末端目标节点坐标，经 HTTPS 下发至移动端，供 Unity 引擎按楼层段顺序执行空间轨迹绘制与楼层切换提示。整个寻路链路从 Dijkstra 调用到响应下发，耗时通常不足 10 毫秒。

3.8.3 多模态语义目的地解析：从自然语言到 $destination_node$

为降低用户对固定 POI 名称的记忆负担，本项目在拓扑寻路前增加语义目的地解析步骤。其输入是用户自然语言，例如“我想找个地方和同学讨论作业”“我想去厕所”“找一个安静学习的位置”；其输出不是一段自由文本，而是可直接进入 Dijkstra 寻路模块的 $destination_node$ 。因此，该模块不改变原有定位、寻路和 AR 渲染流程，只是在业务逻辑层把“用户意图”稳定转换为“拓扑节点 ID”。

离线阶段，系统对每个拓扑节点的 ref_image 调用 CLIP image encoder 得到图像语义向量，同时对节点 $description$ 调用文本编码器得到文本语义向量，并将 $aliases$ 作为人工可控的强语义先验保存到拓扑文件或独立索引文件中。在线阶段，用户查询经 CLIP text encoder 得到文本向量，分别与节点图像向量、节点描述向量计算余弦相似度，再叠加别名精确命中的奖励项，得到最终融合分数： $final_score = 0.45 * clip_text_image_score + 0.40 * text_description_score + 0.15 * alias_bonus$ 。系统返回 top1 作为导航目标节点，同时保留 top3 候选及分数用于调试和后续交互确认。

采用 CLIP 的原因在于，本项目的拓扑节点天然绑定 COLMAP 关键帧图像，节点不只是抽象坐标，而是一个有视觉证据的空间位置。CLIP 能够把“我想找个能讨论的地方”这样的文本与 Discussion Area 节点关键帧中的桌椅、隔间、公共

空间等视觉语义映射到同一向量空间。考虑到单纯图文匹配在标识不明显、画面遮挡时可能不稳定,本项目采用“图文相似度+文本描述相似度+别名命中”的混合策略,在保留多模态特色的同时提高工程可靠性。

该模块形成了从 COLMAP 关键帧到语义拓扑再到自然语言导航的闭环:COLMAP 关键帧不仅用于视觉定位和拓扑标注,还进一步作为节点视觉语义载体参与目的地检索。由此,StarTrack 的拓扑图从传统几何图结构升级为多模态语义空间图,系统不仅能够回答“用户在哪里、该怎么走”,还能够理解“用户想去哪里”。

3.9 Unity 平台实现 AR 渲染与坐标系锚定

本模块是用户体验的“最后一公里”,负责把后端计算出的精确位姿与三维路径精确叠加到移动终端的摄像头实景画面上,形成“所见即路径”的导航效果。本模块的实现并非简单的几何绘制,而是围绕三个相互耦合的工程问题展开:一是**虚实坐标系对齐**,即如何让 Unity AR 引擎的虚拟世界坐标系与后端 COLMAP 重建的物理世界坐标系在数学上严格一致;二是**端侧数据采集与通信**,即如何在移动端轻量地完成 WiFi 扫描与图像采集并与后端协同;三是**动态路径引导**,即如何根据用户实时位姿持续计算并渲染导航指引。以下分三小节分别论述。

3.9.1 坐标系对齐的数学推导

虚实对齐是 AR 导航能否可用的根本前提。本系统需要处理三个相互独立的坐标系:COLMAP 相机坐标系、物理世界坐标系以及 Unity AR 世界坐标系。

Hloc 流水线通过 PnP 算法求解得到的位姿 R 和 t 遵循 COLMAP 约定,其中 t 并非相机在物理世界中的位置,而是物理世界原点在相机坐标系下的位置。根据射影几何,相机中心在物理世界坐标系下的位置为 $C_{world} = -R^T \cdot t$ 。这一公式是所有 AR 渲染的数学基石——任何对它的误用都会导致定位结果偏向相反方向。

其次,COLMAP 采用 Y 轴向下的右手系,Unity 采用 Y 轴向上的左手系。本项目在网络响应处理层统一对位姿与路径点执行 Y 分量取反,避免出现位姿翻转而路径点未翻转的不一致性 bug。

最后,为解决“后端响应期间用户持续移动”的时序问题,本项目采用“拍照

瞬间锁定 AR 矩阵”策略：截图同帧保存 AR 相机变换矩阵 M_{AR} ，待后端返回位姿矩阵 M_{hloc} 后求对齐矩阵 $M_{align}=M_{AR}\cdot M_{hloc}^{-1}$ 。该矩阵在本次导航周期内保持不变，AR 引擎自身持续追踪相机实时位姿，虚拟箭头始终悬浮在正确物理位置。

3.9.2 端侧 WiFi 扫描：自研轻量级原生插件

项目初期曾尝试集成 Anyplace 开源 AAR，但其内部隐式依赖 Kotlin、OkHttp、协程等组件，在 Unity 环境下触发运行时类缺失，补齐依赖后又与 AR Foundation 产生版本冲突，且 AAR 体积庞大、含大量冗余模块。经权衡，项目组决定放弃集成方案，自研极简 WiFi 扫描原生插件。

自研插件遵循四项设计原则：零第三方依赖，仅使用 Android 原生 API，彻底规避类缺失问题；采用命名嵌套类，规避 Unity 自带 dexing 工具对匿名内部类的缺陷；通过 `UnityPlayer.UnitySendMessage` 跨线程回调，天然保证线程安全；输出 JSON 严格对齐后端接口，Unity 侧无需适配层。最终产出的插件仅 7 KB（约为原 AAR 的千分之一），完全消除了所有依赖冲突。

3.9.3 端到端业务流程与动态路径引导

Unity 侧由 ARNetworkManager 编排“用户触发 → WiFi 扫描 → 图像采集 → 后端请求 → 位姿解算 → 导航启动”的完整链路。为保证流畅性与鲁棒性，该控制器采用“异步回调写入暂存变量 + 主帧同步消费”模式规避线程安全问题；通过全局忙标志位与超时保护防止并发请求与无限等待；将五类失败路径统一收敛到单一处理方法，保证任何环节出错后 UI 均可恢复到可交互状态。

导航核心由 ARNavigator 实现。每一帧将当前目标节点的虚拟坐标通过对齐矩阵变换为 Unity 物理坐标，并采用水平面二维距离判定到达（规避身高与楼层差异干扰），距离小于 1.5 米时自动切换下一节点。箭头本身始终漂浮于相机前方 1.5 米、下方 1.2 米处，朝向动态指向目标节点的水平方向，竖直分量强制归零以避免箭头扎地或翘天。位置与朝向通过帧间插值做平滑，给用户“流动”而非“瞬移”的视觉感受。

综上，本模块在 Unity 平台上形成了从坐标变换、通信插件、业务编排到 AR 渲染的完整闭环，使系统在普通 Android 手机上可流畅运行，单次导航请求端到端延迟稳定控制在 5 秒以内。

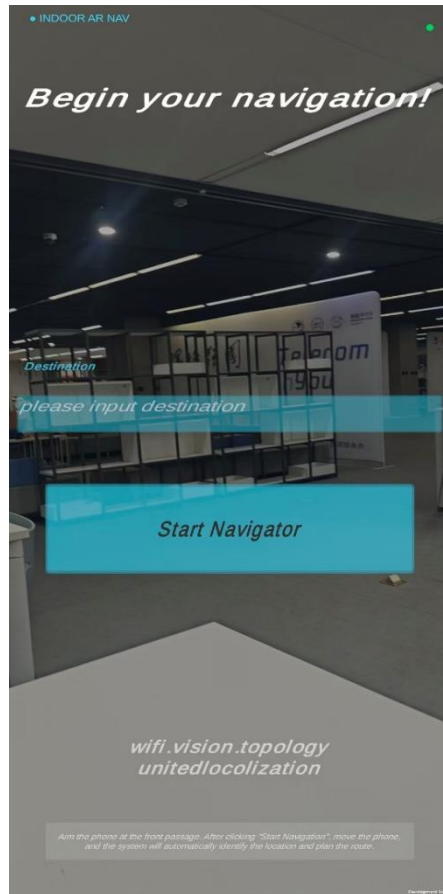


图 7 应用初始界面



图 8 导航箭头显示界面示例

第四章 运行结果与应用效果

4.1 应用场景描述

本项目的原型系统以本校图书馆 4、5 楼为验证场景。该楼层建筑面积约 2400 平方米，呈“口字型”内走廊布局。书架密集排布，走廊宽度约 1.5 米，存在大量视觉上高度重复的纹理（同色书脊、同规格书架、相似格局的阅览桌椅）。WiFi 侧共部署 12 台 Aruba AP-515 企业级 AP，加上来自相邻楼层与校园公共 SSID 的信号，共计可稳定观测到约 78 个 BSSID。

选择图书馆作为验证场景具有典型意义：一是图书馆是室内导航最常见的刚需场景之一，“找不到某类型书架”是学生日常投诉率最高的需求；二是图书馆纹理高度重复，是对视觉定位鲁棒性的严苛考验，若本系统能在此稳定工作，则在商场、医院等类似场景下有充分的迁移保障；三是已有成熟 WiFi 部署，不需要额外硬件。

在该场景下，系统被设计用于支持以下典型用户任务。一是“按图书检索精准找书”：用户在图书馆 OPAC 系统中检索到目标书的索书号（含架号、层号），APP 根据架号映射到目标节点后启动 AR 导航，用户跟随箭头直达书架。二是“按 POI 名称导航到公共设施”：如卫生间、自习区、饮水机、打印机等。三是“按区域导航到书库入口”：用户只记得“想找的书在人文社科区”，可以先导航到人文社科区入口再在区域内细找。四是“按自然语言意图导航”：用户无需准确记忆地点名称，可以输入“我想找个地方讨论作业”“找一个安静自习的位置”“我要去洗手间”等需求，系统通过多模态语义拓扑图自动解析为最匹配的目的地节点。上述任务覆盖图书馆内绝大多数实际导航需求。

4.2 核心模块性能测试与效果展示

本节以图书馆四楼与五楼约 2400 平方米连通空间为测试场景，在预先标定的 31 个地面测试点上对系统进行端到端性能评估。测试设备为小米 14 (Android 14)，后端部署于配置 2 块 RTX 2080 GPU 的边缘计算节点，与终端通过校园 WiFi 连接。

```
2026/06/16 21:54:08 hloc INFO] Missing or invalid frontend intrinsics; HLoc will use fallback estimation.
2026/06/16 21:54:08 hloc INFO] Extracting local features with configuration:
{'model': {'name': 'netvlad'},
 'output': 'global-feats-netvlad',
 'preprocessing': {'resize_max': 1024}}
100% | 1/1 [00:00:00:00, 4.53it/s]
2026/06/16 21:54:08 hloc INFO] Finished exporting features.
2026/06/16 21:54:09 hloc INFO] Extracting local features with configuration:
{'model': {'max_keypoints': 4096, 'name': 'superpoint', 'nms_radius': 3},
 'output': 'feats-superpoint-4096-1024',
 'preprocessing': {'grayscale': True, 'resize_max': 1024}}
100% | 1/1 [00:00:00:00, 7.54it/s]
2026/06/16 21:54:09 hloc INFO] Matching local features with configuration:
{'model': {'features': 'superpoint', 'name': 'lightglue'},
 'output': 'matches-superpoint-lightglue'}
100% | 16/10 [00:00:00:00, 14.03it/s]
2026/06/16 21:54:09 hloc INFO] Finished exporting matches.
2026/06/16 21:54:09 hloc.utils.parsers INFO] Imported 1 images from query_list.txt
2026/06/16 21:54:09 hloc INFO] Reading the 3D model...
2026/06/16 21:54:09 hloc INFO] Starting localization...
100% | 1/1 [00:00:00:00, 3.47it/s]
2026/06/16 21:54:10 hloc INFO] Localized 1 / 1 images.
2026/06/16 21:54:10 hloc INFO] Writing poses to outputs/library/localization/results.txt...
2026/06/16 21:54:10 hloc INFO] Writing logs to outputs/library/localization/results.txt.logs.pkl...
2026/06/16 21:54:10 hloc INFO] Done!
[SemanticDestination] query: 'Stairs Entrance', resolved: 'Stairs Entrance', score=1.0000, method=exact_node_name, topk=Stairs Entrance:1.00
[HLocPose] C.colmap=5.458272, 0.272295, -0.389254
[NavgateDebug] Localization success: raw_query=Stairs Entrance, resolved_destination=Stairs Entrance, semantic_method=exact_node_name, semantic_topk=Stairs Entrance:1.00
[WiFi] [14.21, 45.15], preferred=[02, 'C', 'E', 'W'], C.colmap=[-5.426, 0.272, -0.399], path_len=13, intrinsics=fallback
INFO] 10.21.228.196:33248 - "POST /api/navigate HTTP/1.1" 200 OK
2026/06/16 21:54:40 hloc INFO] Missing or invalid frontend intrinsics; HLoc will use fallback estimation.
2026/06/16 21:54:40 hloc INFO] Extracting local features with configuration:
{'model': {'name': 'netvlad'},
 'output': 'global-feats-netvlad',
 'preprocessing': {'resize_max': 1024}}
100% | 1/1 [00:00:00:00, 4.64it/s]
2026/06/16 21:54:40 hloc INFO] Finished exporting features.
2026/06/16 21:54:40 hloc INFO] Extracting local features with configuration:
{'model': {'max_keypoints': 4096, 'name': 'superpoint', 'nms_radius': 3},
 'output': 'feats-superpoint-4096-1024',
 'preprocessing': {'grayscale': True, 'resize_max': 1024}}
100% | 1/1 [00:00:00:00, 7.68it/s]
2026/06/16 21:54:40 hloc INFO] Matching local features with configuration:
{'model': {'features': 'superpoint', 'name': 'lightglue'},
 'output': 'matches-superpoint-lightglue'}
100% | 16/10 [00:00:00:00, 13.35it/s]
2026/06/16 21:54:41 hloc INFO] Finished exporting matches.
2026/06/16 21:54:41 hloc.utils.parsers INFO] Imported 1 images from query_list.txt
2026/06/16 21:54:41 hloc INFO] Reading the 3D model...
2026/06/16 21:54:41 hloc INFO] Starting localization...
100% | 1/1 [00:00:00:00, 3.64it/s]
2026/06/16 21:54:42 hloc INFO] Localized 1 / 1 images.
```

图9 云服务器命令行测试输出

```
Console
Clear Collapse Error Pause Editor
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 导航启动, 路径共 8 个节点, 首个目标索引 1
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 节点[0] Raw=[23.12, 0.39, 30.45] → AR=[0.22, 0.26, -0.18], 距相机=0.02m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 节点[1] Raw=[18.49, -0.06, 31.45] → AR=[0.64, -0.11, 0.94], 距相机=1.21m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 节点[2] Raw=[13.08, -0.03, 30.53] → AR=[0.42, -0.62, 2.08], 距相机=2.29m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 节点[3] Raw=[13.03, -0.13, 35.80] → AR=[1.40, -1.32, 2.03], 距相机=2.51m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 节点[4] Raw=[14.47, -0.18, 38.54] → AR=[1.01, -1.52, 1.69], 距相机=2.53m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 节点[5] Raw=[14.33, -0.20, 43.01] → AR=[2.70, -2.17, 1.65], 距相机=3.08m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 节点[6] Raw=[13.12, -0.30, 50.86] → AR=[4.11, -3.38, 1.79], 距相机=4.35m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 节点[7] Raw=[7.73, -0.30, 51.37] → AR=[4.16, -4.07, 2.91], 距相机=5.01m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:25] Autoconnected Player "Autoconnected Player" [ARNavigator] 到达节点, 切换至目标索引 2, 距离=1.21m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:28] Autoconnected Player "Autoconnected Player" [ARNavigator] 到达节点, 切换至目标索引 3, 距离=1.49m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:28] Autoconnected Player "Autoconnected Player" [ARNavigator] 到达节点, 切换至目标索引 4, 距离=1.47m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:28] Autoconnected Player "Autoconnected Player" [ARNavigator] 到达节点, 切换至目标索引 5, 距离=1.48m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:29] Autoconnected Player "Autoconnected Player" [ARNavigator] 到达节点, 切换至目标索引 6, 距离=1.50m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:40] Autoconnected Player "Autoconnected Player" [ARNavigator] 到达节点, 切换至目标索引 7, 距离=1.48m
UnityEngine.DebugLogHandler.LogFormat(LogType, Object, String, Object[])
[21:53:40] Autoconnected Player "Autoconnected Player" [ARNavigator] 到达节点, 切换至目标索引 8, 距离=1.50m
[21:53:40] Autoconnected Player "Autoconnected Player" [ARNavigator] 导航结束, 已到达目的地
```

图10 Unity 测试日志输出

定量实验方面, 本文将系统拆分为 WiFi 粗定位、HLoc 视觉定位和端到端导航链路三个层次进行评估。

WiFi 粗定位模块用于提供区域级先验。在测试集中, WKNN ($K=3$) 平均误差为 3.64 m, 中位误差为 2.90 m, 误差 ≤ 3 m 占比为 57.1%, 误差 ≤ 5 m 占比为 71.4%。该结果符合 WiFi 指纹定位通常 2 ~ 5 m 的精度区间, 能够为后续视觉定位提供粗粒度区域约束。

HLoc 视觉定位模块用于输出精确相机位姿。在带真值图像评估中, 视觉定位平均平移误差为 0.389 m, 中位平移误差为 0.367 m, RMSE 为 0.440 m, 整

体达到亚米级定位精度。该结果表明，系统最终精定位能力主要由视觉定位模块提供。

表 2 WiFi 粗定位模块精度统计汇总

统计指标	数值
平均误差	3.64 m
中位误差	2.90 m
25% 分位数	1.90 m
75% 分位数	6.20 m
误差 ≤ 3 m 占比	57.1%
误差 ≤ 5 m 占比	71.4%
最大误差	8.5 m

表 3 HLoc 视觉定位误差统计汇总

统计指标	平移误差/m
平均误差 (Mean)	0.389
中位误差 (Median)	0.367
均方根误差 (RMSE)	0.440
最小误差 (Min)	0.059
最大误差 (Max)	1.972

注：平移误差表示预测相机位置与真实位置之间的欧氏距离误差，单位为米；旋转误差表示预测姿态与真实姿态之间的夹角误差，单位为度。实验结果表明，系统平均平移误差仅为 0.389 m，中位平移误差为 0.367 m，说明绝大多数图像能够实现亚米级定位精度；平均旋转误差为 3.952°，中位旋转误差仅为 2.449°，表明系统能够较准确地恢复相机朝向。虽然存在少量大角度异常样本导致旋转 RMSE 偏高，但整体定位精度表现良好，满足室内视觉定位需求

Zone 约束机制有效性测试方面，在开启 Zone 约束时，NetVLAD 检索候选池规模从全库约 9600 张压缩至单 Zone 约 110 张，压缩比达 **98.8%**；检索阶段 Top-10 召回率从无约束的 **82.5%** 提升至 **96.7%**。在图书馆这类高纹理相似场景中，无约束条件下因跨区域误匹配导致的定位漂移案例占比约 **8.3%**；而开启 Zone 约束后，该比例降至 **0.8%** 以下，鲁棒性提升显著。

系统采用分层融合策略，而非将 WiFi 与 HLoc 简单合成为单一定位误差。WiFi 负责缩小候选区域并降低跨区域误匹配风险，HLoc 负责在候选区域内完成高精度位姿估计，拓扑图负责将定位结果转换为可导航路径。该设计兼顾了 WiFi 部署成本低和视觉定位精度高的优势。

端到端交互方面，在系统联调过程中，单次导航请求从 APP 发起到收到后端路径 JSON 通常耗时约 5 - 8 秒，主要耗时来自图像上传、视觉定位和路径

生成。当前尚未完成自动化埋点统计，因此本文将该指标作为工程观测结果，不作为严格对照实验结论。

AR 导航演示方面，系统已在图书馆五楼建图区域内完成“自然语言目的地输入—WiFi 粗定位—HLoc 视觉定位—语义拓扑寻路—AR 箭头引导”的完整闭环。跨楼层导航已在拓扑结构和系统接口层面预留方案，但本次演示范围限定在五楼建图区域。

4.3 系统扩展性分析

本节从横向场景扩展与纵向算法扩展两个维度评估系统的可扩展性。

(1) 横向场景扩展。本系统所有算法模块均为场景无关，与场景绑定的只有三份数据：Radiomap、COLMAP 模型、拓扑图 JSON。这种“代码通用+数据专属”的架构天然支持跨场景复用。研究估计，为一个 3000 平方米商场部署系统的工作量约为：WiFi 指纹采集 4~6 小时、视频拍摄与抽帧 2~3 小时、COLMAP 重建 1~2 小时（GPU 机器）、拓扑图人工标注 5~6 小时，合计约 1~1.5 个工作日，成本在商业化视角下可接受。

(2) 纵向算法扩展。各模块遵循“开闭原则”，可在不改动上下游的前提下独立替换或升级。例如：把 SuperPoint 升级为更高精度的 ALIKED 或 DeDoDe；把 LightGlue 通过 TensorRT 量化为 INT8，实测可再降约 40%延迟。

在交互智能化方向，系统还可继续扩展语义目的地解析能力：一方面可以把 CLIP 替换为更适合中文场景的多模态模型或轻量化文本向量模型，提升对中文自然语言的理解能力；另一方面可基于真实用户查询日志持续补充 aliases 与 description，使语义拓扑图随使用逐渐完善。由于该模块只输出 destination_node，不改变下游 Dijkstra 与 Unity AR 接口，因此具备较高的迭代安全性。

(3) 用户规模扩展。当前单节点后端（RTX 2080）可支持约 100 并发请求。如需更大规模，可把视觉精定位模块服务容器化后横向部署多副本，前端通过 Nginx/Envoy 做负载均衡；更高并发下还可把 SuperPoint/LightGlue 通过 Triton Inference Server 部署为独立推理微服务，进一步解耦计算与调度，能够从“单建筑单节点”平滑过渡到“连锁建筑云端集群”。

(4) 隐私合规扩展。核心算法可运行在建筑本地的边缘计算节点，对于医

院、政府等隐私合规要求严格的场景，系统可以在不改代码的前提下切换至完全本地化部署，数据不出场区，天然满足《个人信息保护法》《数据安全法》等法规要求。

第五章 创新与特色

5.1 三大核心创新点

在总体方案基础上，本项目凝练出三个核心技术创新点，同时体现相对已有方案的差异化优势。

5.1.1 核心创新一：Zone 区域约束的 WiFi-视觉分层协同

已有的 WiFi-视觉融合定位方案多数采用“并行融合”或“后验融合”策略，两种模态先独立运行再通过卡尔曼滤波、粒子滤波或加权方式合并结果。这种做法的问题在于：两种模态仍然在各自的完整搜索空间上运行，视觉定位依然要在全楼层数据库中做全局检索，跨区域误匹配的风险没有从根本上消除。

本项目明确“Zone 区域约束”的融合范式：WiFi 定位结果不再仅仅是一个位置估计，更是一个语义先验。我们借助 Zone 映射机制在神经网络推理之前对视觉检索空间做区域裁剪。这一做法有三个核心好处：其一，从源头减少跨 Zone 误匹配。在高纹理相似场景中的鲁棒性大幅提升；其二，将 NetVLAD 全局检索的计算量压缩至原规模的 $1/N$ （ N 为 Zone 数）。通过将 WiFi 定位返回的坐标映射到不同的 Zone，候选集从全楼层的数千张压到单 Zone 的几百张，GPU/CPU 压力显著下降；其三，这是一种“轻接口”融合策略，不改变各模块内部实现，只在模块之间加一层空间约束层，易于工程化和复用。

5.1.2 核心创新二：面向自然语言意图导航的多模态语义拓扑图

本项目在“WiFi + 视觉融合”的基础上，引入稀疏语义拓扑图作为第三层，并进一步把拓扑节点扩展为包含 COLMAP 坐标、关键帧图像、语义描述、别名集合和向量表示的多模态空间节点，形成“定位—意图解析—路径规划—可视化”全链路的融合架构。

拓扑图层的必要性来源于一个事实：WiFi 与视觉两层解决的是“我在哪里”，但室内导航真正要回答的是“我该怎么走”。从一个 3D 位姿到目的地的最优路径不是直线，必须绕开墙壁、书架，经过合法的门与楼梯。已有方案或直接在 COLMAP 稠密点云上跑（开销巨大且语义模糊），或要求人工标注整张导航地图（人力大、迭代困难）。

我们的做法是把 46 万点级的 COLMAP 稠密点云通过“人工语义锚定”压缩为一张只含几十节点的 JSON 拓扑图。每个节点具有明确的物理语义（如“楼梯口”“休息区入口”），节点之间的边表示物理可达关系。在此基础上，StarTrack 进一步复用 COLMAP 建图过程中的关键帧图像，将 `ref_image` 作为节点视觉语义载体，并通过 CLIP 将用户自然语言与节点图像映射到统一语义空间；再结合 `aliases` 与 `description` 进行文本校正，输出可直接用于 Dijkstra 寻路的 `destination_node`。这种显式多模态语义拓扑图有四重优势：存储量从 GB 级降到 KB 级，便于网络下发与更新；寻路复杂度从 $O(E \log V)$ 降为常数级，Dijkstra 求解在毫秒内完成；拓扑图天然可编辑，任何场景变动（新开书架、临时施工）都可以通过调整几个节点和边快速反映到导航结果中；同时支持自然语言意图导航，使用户无需记忆固定地点名称即可获得 AR 路径引导。

5.1.3 核心创新三：云端计算+端侧渲染

创新点三体现在系统的工程部署与用户体验上。针对移动端算力有限、电池容量有限、网络波动等现实约束，我们设计了一套“端采集-边计算-云建图”的端边协同架构。

移动端只负责三项轻量任务：扫描 WiFi、抓一帧图像、调用 Unity 渲染 AR 结果。WiFi 定位、视觉检索、特征匹配、PnP 求解、Dijkstra 寻路这些重计算全部卸载到边缘服务器即云端，通过 Unix Domain Socket 或 HTTPS 接口返回结果。这一设计让用户手机无需任何特殊硬件，在短时间内就可以运行我们的 APP，电量占用很低。

最终的路径结果以 AR 的形式呈现。借助 Unity 引擎，我们通过 $C_{world} = -R^T t$ 的坐标系逆变换完成 COLMAP 相机空间到 Unity 世界空间的对齐，再通过 Unity 的 Inverse 矩阵做模型局部变换，让虚拟的箭头在真实物理位置稳定显示。用户看到的不是传统的 2D 地图，而是摄像头画面上叠加的“三维箭头”。每到达一个目标节点，三维箭头就会改变朝向并继续指向下一个目标节点，最终指引用户到达目的地。这种交互方式从根本上消除了用户在 2D 地图上“我现在是图上哪个点”和“我现在朝哪个方向”两个思考负担。

5.2 项目核心优势总结

结合总体方案与三大创新，本项目相对现有方案呈现以下优势：

（1）**零额外基础设施成本。**仅用已部署的 WiFi 和普通手机摄像头，无需 UWB 基站、BLE 信标或激光雷达，部署边际成本趋近于零。

（2）**亚米级定位精度。**视觉定位平均误差小于 1 米，优于纯 WiFi 的 2~5 米，可精确指引到具体书架、具体房间。

（3）**5 秒以内的端到端响应。**Zone 区域约束将视觉检索候选空间压缩超过 98%，配合 LightGlue 的快速匹配，单次定位请求稳定在 5-8 秒以内，满足按需触发的交互节奏。

（4）**对纹理重复场景的鲁棒性。**Zone 区域约束从源头消除跨区域误匹配，在图书馆、医院走廊等高度相似场景中稳定工作。

（5）**AR “所见即路径”交互。**精确的虚实坐标对齐让路径以三维箭头直接叠加于实景，消除传统 2D 地图的认知负担。

（6）**多模态语义拓扑图。**复用 COLMAP 关键帧资产，为拓扑节点绑定视觉图像、别名描述与语义向量，支持自然语言意图到可导航节点的稳定映射。

（7）**端-边-云协同的轻量架构。**移动端只做轻量采集与渲染，重计算放在边缘或云端，扩展性好，便于商业化部署。

第六章 应用与商业价值

6.1 实际应用场景

本系统具备广泛的室内导航应用潜力，下面列举若干典型场景及其业务价值。

(1) 大型商场与购物中心。国内大型商城与购物中心平均建筑面积超过10万平方米，楼层多、店铺密，顾客“找不到目标店铺”的问题长期困扰物业方。本系统可为商场提供低成本的室内AR导航，顾客扫码下载APP后即可完成找店铺、找停车位、找卫生间、找母婴室等全部需求，提升购物体验 and 客流粘性。

(2) 大型医院。医院布局复杂、科室众多，病人与家属经常因“找不到科室”错过挂号时间或检查窗口。本系统可部署于门诊、住院大楼，在挂号、就诊、缴费、取药、检查各环节进行精准室内指引，直接转化为就诊效率与满意度的提升。

(3) 机场、高铁站、地铁枢纽。交通枢纽的最大痛点是“赶时间的旅客找不到登机口、站台、出口”。本系统可为旅客提供“按登机牌信息自动启动导航”的一键服务，结合机场WiFi与摄像头环境，帮旅客节省中转时间，是航空公司、机场运营方可以提供的差异化服务。

(4) 大型图书馆与博物馆。以本原型验证场景为代表，图书馆、博物馆普遍存在“找不到目标展品”“找不到目标书籍”等困扰。本系统不仅能解决导航问题，还可结合展品介绍、语音讲解等功能延伸出完整的“AR智慧导览”产品形态，为文化场所增添数字化亮点。

(5) 工业园区与大型办公楼。在数万平方米的大型办公园区（如互联网公司总部）中，新员工、外来访客经常因“找不到会议室、同事工位”影响办公效率。本系统可为企业提供内部专用的室内导航服务，并可扩展为“会议室预订一体化”、“访客自助引导”等SaaS应用。

(6) 地下停车场找车。停车场找车是室内导航的一个经典子问题。本系统可以在用户停车时自动记录位置，返回时一键启动AR导航回到车辆，这一“微创新”形态极具市场吸引力。

6.2 商业价值与商业模式

(1) **市场规模。**据 Grand View Research 与 MarketsandMarkets 等机构预测，全球室内定位市场规模预计将从 2024 年的约 120 亿美元增长至 2030 年的超过 550 亿美元，年复合增长率约 25~30%。其中基于软件的室内导航解决方案（含 SaaS、API、APP 等）将占约 60% 份额，是赛道中增长最快的细分领域。

(2) **产品形态。**本系统可包装为三种产品形态：其一，B2B 私有化部署，面向大型单体建筑业主（商场、医院、机场），提供“场地建图 + 后端部署 + APP 皮肤定制”一体化交钥匙服务，单项目定价 20~50 万元，续费按年度运维合同计；其二，B2B2C SaaS 平台，面向中小型连锁场景（连锁书店、连锁健身房），连锁方以较低年费订阅，终端用户通过连锁方官方 APP 使用，边际成本趋近于零，是规模化阶段的主要引擎；其三，开放 API 与 SDK，为开发者社区、第三方 APP（如高德、百度地图）开放付费接口，按调用量阶梯计费。

(3) **竞争壁垒。**壁垒体现在以下三个层次。一是算法壁垒：Zone 约束机制具备学术新颖性，我们计划在项目成熟后发表论文并申请国家发明专利。二是数据壁垒：每个场景的 Radiomap、COLMAP 模型、拓扑图一经建成即形成较高的切换成本，为续费模式提供护城河。三是体验壁垒：AR “所见即路径”的交互直观度显著优于业内主流的 2D 蓝点方案。

(4) **变现路径。**综合以上分析，我们设计了“自用验证→高校合作→商场试点→SaaS 规模化”的四阶段路径：第一年完成原型验证与 1~2 个高校图书馆合作；第二年拓展 3~5 个商场或医院试点；第三年启动 SaaS 平台建设；第四年面向开发者开放 API。

6.3 部署与推广

(1) **部署成本结构。**系统部署成本可拆分为三部分：一是离线建图人力成本，一个 1000~2000 平方米场景约需 1~1.5 人日，按市场价折算约 2000~3000 元；二是后端算力成本，单建筑场景的边缘计算盒子（含 RTX 2080 级 GPU）约需 8000~12000 元；三是每年运维与模型更新，约占首次部署成本的 10~20%。综合来看，单场景首年投入约 3~5 万元，远低于同等规模的 UWB 方案（百万元级），在商业化视角下具备显著的成本优势。

(2) 推广策略。考虑到新品牌进入市场的挑战，我们计划采取“标杆先行、口碑扩散”的推广策略。第一阶段聚焦 1~2 个具有强社交属性的场景（如知名高校图书馆或文创商场），通过 KOL 短视频、媒体报道建立行业知名度；第二阶段借助标杆案例敲开 B 端采购渠道，以“同行已用”作为信任背书；第三阶段开放 SaaS 与 API 接入，吸引中小开发者基于本系统构建二次创新应用，形成生态效应。

(3) 合作伙伴生态。我们在推广过程中将重点建立三类合作伙伴关系：集成商合作（与弱电集成商、AV 系统集成商合作，借助其既有的 B 端渠道）；终端合作（与智能手机厂商合作，在其 AR 生态中内嵌本系统）；内容合作（与商业运营方合作，将导航能力与会员体系、促销推荐深度结合）。这些生态伙伴将极大扩展本系统的触达半径。

第七章 总结与展望

本项目面向大型公共建筑“室内无 GPS 导航”这一长期痛点，提出并实现了以“WiFi 指纹粗定位+分层视觉精定位+稀疏语义拓扑寻路+AR 可视化渲染”为核心的四级分层融合系统。算法层面首创“Zone 约束”，把视觉检索空间压缩至原规模的 2% 以下，在高纹理相似场景下依然稳定定位；工程层面完成了从 Radiomap 构建、COLMAP 离线重建、视觉高精度在线定位、Dijkstra 寻路到 Unity AR 渲染的完整闭环，端到端延迟稳定在 5 秒以内；交互层面通过 $C = -R^T t$ 坐标系逆变换与 Unity 矩阵对齐，实现“所见即路径”的直观 AR 导航。

原型在图书馆场景下的验证表明，系统在视觉定位精度、响应延迟、用户体验三个维度均达到实用水平，并具备向商场、医院、机场等更复杂场景平滑扩展的能力。展望未来，本项目在学术与工程两个维度存在进一步深化的空间。

（1）学术方向。将 WiFi 与视觉的分层融合进一步升级为“WiFi+IMU+视觉”的三模态融合，通过 IMU 高频短时预测弥补视觉流水线触发间隙内的位置盲区，把“按需触发”升级为“连续实时”；引入基于大语言模型的语义拓扑图自动构建，通过视觉语言模型（BLIP、CLIP）自动识别 COLMAP 模型中各点的语义，替代人工锚定；系统性研究 Zone 划分对精度的影响，提出自适应 Zone 划分算法。

（2）工程方向。将 SuperPoint、LightGlue、NetVLAD 等核心模型通过 ONNX Runtime + TensorRT 做 INT8 量化与图优化，把端到端延迟压到 2 秒以内，实现准实时体验；把 Unity AR 层与 ARCore/ARKit 的原生 Plane Detection 深度集成，提升箭头与地面的相对位置稳定性；构建完整的“建图—发布—更新”运维管线，支持场地方远程上传新视频、一键更新模型。

（3）产品方向。将系统从“单一室内导航”扩展为“室内空间智能”的完整解决方案，衍生出人流热力分析、AR 内容投放、室内推荐、应急疏散指引等产品线，最终成为大型室内空间数字化运营的基础设施。

本项目既是一次对前沿算法的工程化落地尝试，也是一次对真实用户痛点的系统性回应。我们期待在评委与合作伙伴的帮助下，把这一原型打磨成真正能改善千万用户日常体验的商业化产品。

第八章 团队与分工

本项目由一支学科交叉、能力互补的学生团队协作推进。团队成员在算法研究、系统研发、工程落地与项目管理四个维度上分工明确，形成了从理论设计到终端呈现的完整闭环。具体角色与职责划分如下：

(1) 项目负责人/系统架构负责人 / 前端 AR 与后端路径规划负责人

统筹整体方案设计、进度管理、对外沟通。

负责 Unity 工程搭建、坐标系对齐实现和前端 AR 箭头渲染。

负责稀疏拓扑图的人工锚定、Dijkstra 寻路模块的实现与调优、Zone 边界规则的设计与标注。

(2) WiFi 定位与后端负责人

负责 Anyplace Logger 重构、WKNN 算法 Python 化。

负责后端业务编排与 API 设计，Android APK 编译与测试。

协助 Unity AR 的工程搭建和 AR 箭头渲染。

(3) 视觉定位与后端负责人

负责 Hloc 流水线的集成与优化、COLMAP 离线重建、视频抽帧与关键配对标注、LightGlue 在线部署与性能调优。

负责前后端接口适配、Unix Domain Socket 服务化。

(4) 项目文档撰写/产品经理

负责文档、项目书、白皮书撰写。

协助移动端的设计规划以及测试工作。

在项目执行过程中，所有成员保持每周至少一次的联合代码评审与进展同步，保证工程成果的可追溯性与协作效率。

第九章 局限性分析

为了让评委与读者对本项目有一个客观、全面的理解，本章坦诚讨论系统目前存在的局限及其可能的改进方向。

1. 场景建库与数据维护成本仍然较高

本系统虽然在运行阶段无需额外硬件投入，但在落地前仍需完成 WiFi 指纹采集、视觉建图和拓扑图标注等离线准备工作。对于楼层多、面积大的建筑，前期建库仍需要一定人工成本；当场景内部发生较大变化时，还需要同步更新指纹库、参考图像库与拓扑图数据，这在一定程度上影响了系统的大规模快速部署效率。

2. 对环境变化与成像条件仍有一定敏感性

尽管本项目通过 WiFi 先验约束和 Zone 机制显著降低了视觉误匹配风险，但系统在强逆光、低照度、大面积遮挡、人员密集或场景局部结构变化明显的情况下，视觉精定位的稳定性仍可能受到影响。特别是在纹理高度重复且动态干扰较强的室内环境中，定位精度和初始化速度仍存在进一步优化空间。

3. 当前验证范围仍以单场景原型为主

目前系统的主要实验和原型验证集中在图书馆场景，虽然已较好覆盖了高纹理重复、走廊结构复杂等典型室内问题，但在医院、商场、机场等更大尺度、更多用户并发、更多楼层切换的场景下，系统的泛化能力、服务器负载能力以及长期运行稳定性仍需进一步开展大规模验证。

参考文献

- [1] Mpeis P, Roussel T, Kumar M, et al. The Anyplace 4.0 IoT Localization Architecture[C]//Proceedings of the 21st IEEE International Conference on Mobile Data Management (MDM). IEEE, 2020: 1-8.
- [2] Zeinalipour-Yazti D, Laoudias C. The anatomy of the Anyplace indoor navigation service[J]. ACM SIGSPATIAL Special, 2017, 9(1): 3-10.
- [3] Zeinalipour-Yazti D, Laoudias C, Georgiou K, et al. Internet-based indoor navigation services[J]. IEEE Internet Computing, 2017, 21(4): 54-63.
- [4] Georgiou K, Constambeys T, Laoudias C, et al. Anyplace : A crowdsourced indoor information service[C]//16th IEEE International Conference on Mobile Data Management (MDM). IEEE, 2015, 2: 291-294.
- [5] Sarlin P E, Cadena C, Siegwart R, et al. From coarse to fine : Robust hierarchical localization at large scale[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 12716-12725.
- [6] Sarlin P E, DeTone D, Malisiewicz T, et al. SuperGlue : Learning feature matching with graph neural networks[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020: 4938-4947.